TRƯỜNG TH & THCS, THPT FPT BẮC NINH

FPT SCHOOLS

LẬP TRÌNH PYTHON

Giáo viên: Nguyễn Tài Sỹ

Email: Synt6@fpt.edu.vn

Sđt: 0967698296

MỤC LỤC

Nội dung

Bài 1. Giới thiệu python

Bài 2. Biến và kiểu dữ liệu trong python

Bài 3. Cấu trúc điều kiện

Bài 4. Vòng lặp While

Bài 5. Vòng lặp xác định For

Bài 6 và 7. Ôn tập

Bài 8: Tìm hiểu về Chuỗi (String) trong Python

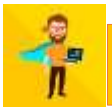
Bài 9: List trong Python

Bài 10: Hàm trong Python

BÀI SỐ 1. GIỚI THIỆU VỀ PYTHON

Nội dung bài học

- **Giới thiệu khái quát về Python**
- **Cách tải và cài đặt Python**
- **Giới thiệu một số công cụ lập trình Python**
- **Giới thiệu công cụ hỗ trợ lập trình python: PyCharm và Wingide**
- **Sử dụng hàm print()**



1. Giới thiệu khái quát về Python

1.1. Tác giả và lịch sử phát triển

- Python ra đời năm: 1990 do tác giả: Guido van Rossum



(Hình 1. Tác giả Guido van Rossum) - Các phiên bản:

+ Python 1: các phiên bản 1.x, từ 1990

+ Python 2: các phiên bản 2.x, từ 2000

+ Python 3: Python3000 hoặc Py3K, gồm các bản 3.x, từ 2008



Hình 2- Logo Ngôn ngữ python

1.2. Đặc điểm Python

- **Python dễ dàng kết nối với các thành phần khác**

Python có thể kết nối với các đối tượng COM, .NET (Ironpython, Python for .net), và CORBA, java… Python cũng được hỗ trợ bởi Internet Communications Engine (ICE) và nhiều công nghệ kết nối khác.

- **Python là ngôn ngữ có khả năng chạy đa nền tảng**

Python có cho mọi hệ điều hành: Windows, Linux/Unix, OS/2, Mac, Amiga, và những hệ khác. Thậm chí có cả những phiên bản chạy trên .NET, máy ảo Java, và điện thoại di động Nokia Series 60. Với cùng một mã nguồn sẽ chạy không khác nhau trên mọi cài đặt.

- **Python rất đơn giản và dễ học**

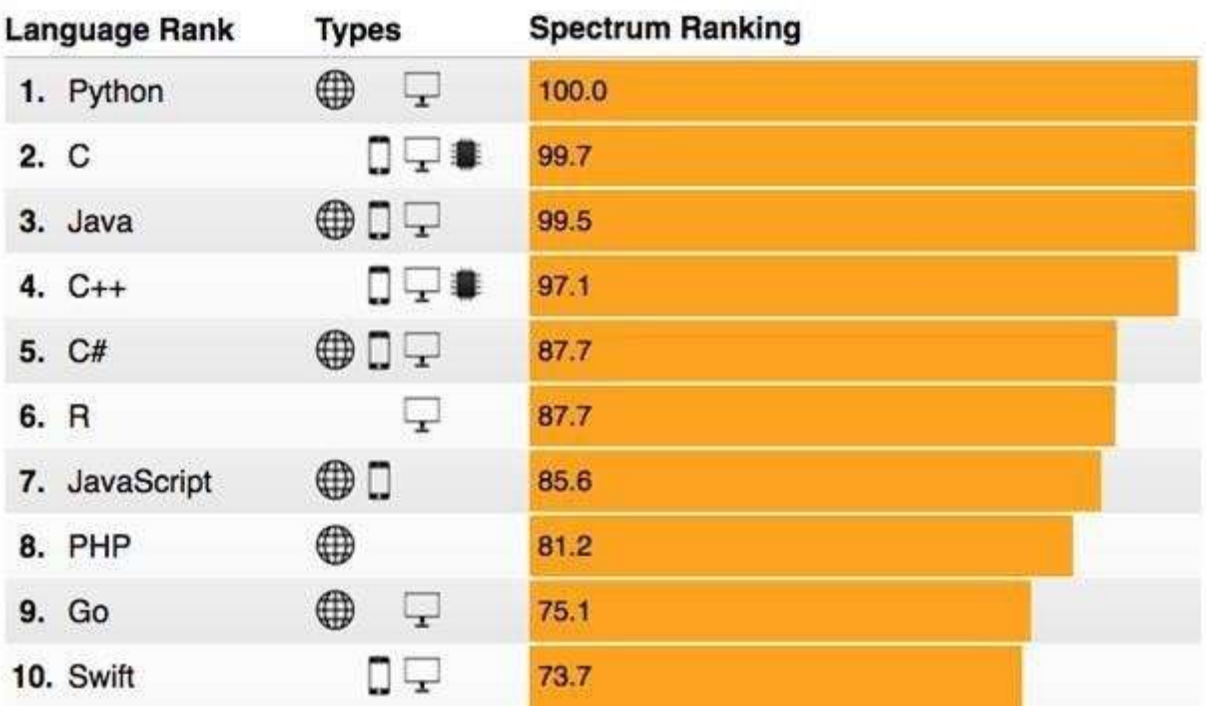
Python có cộng đồng lập trình rất lớn, được chia sẻ rất nhiều thông tin hữu ích. Python cũng có kho tài liệu đầy đủ, vừa tồn tại trong bản thân ngôn ngữ lẫn trên mạng. Những tài liệu trên mạng nhắm vào cả lập trình viên chuyên nghiệp lẫn người mới. Tất cả đều được thiết kế nhằm giúp bạn sản xuất nhanh hơn.

- **Python là ngôn ngữ mã nguồn mở**

Cài đặt Python dùng giấy phép nguồn mở nên được sử dụng và phân tối tự do, ngay cả trong việc thương mại. Giấy phép Python được quản lý bởi Python Software Foundation.

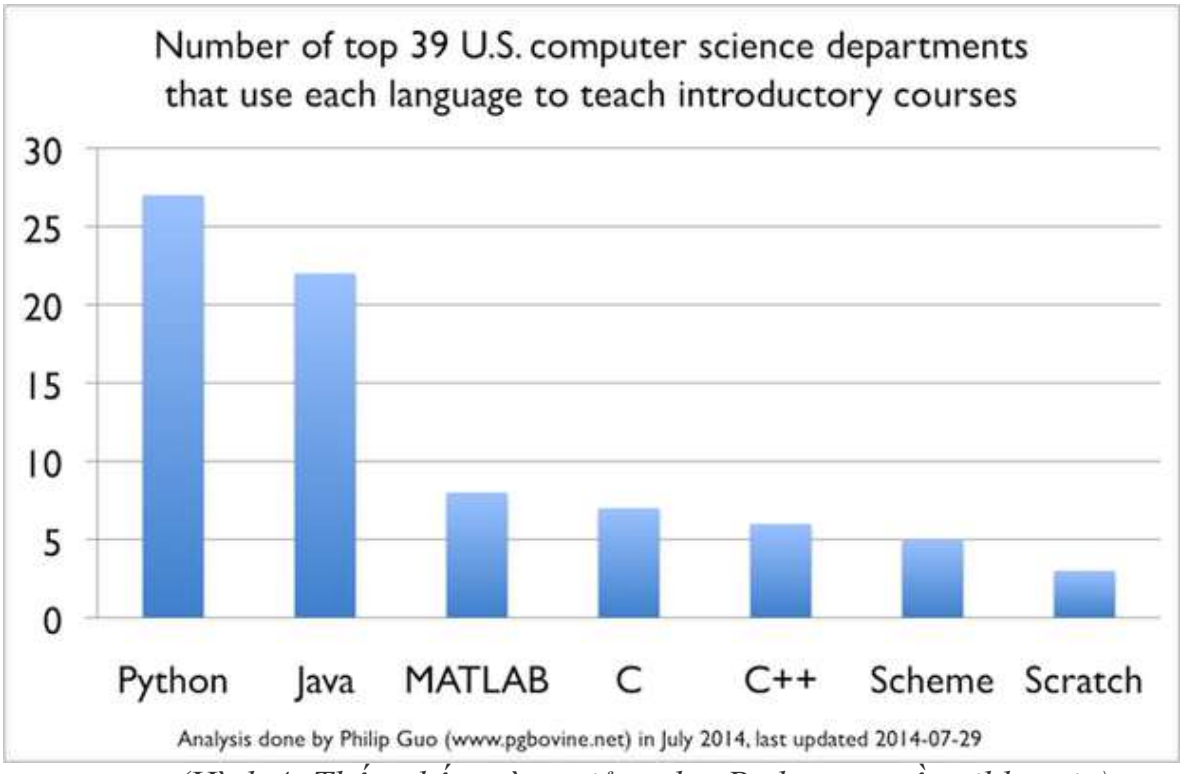
1.3. Lĩnh vực áp dụng Python

Hiện nay ngôn ngữ Python được xếp hạng thứ 1 trong Top 10 các ngôn ngữ lập trình phổ biến nhất đang sử dụng:



(Hình 3 – Xếp hạng ngôn ngữ lập trình – 2017, nguồn http://spectrum.ieee.org/)

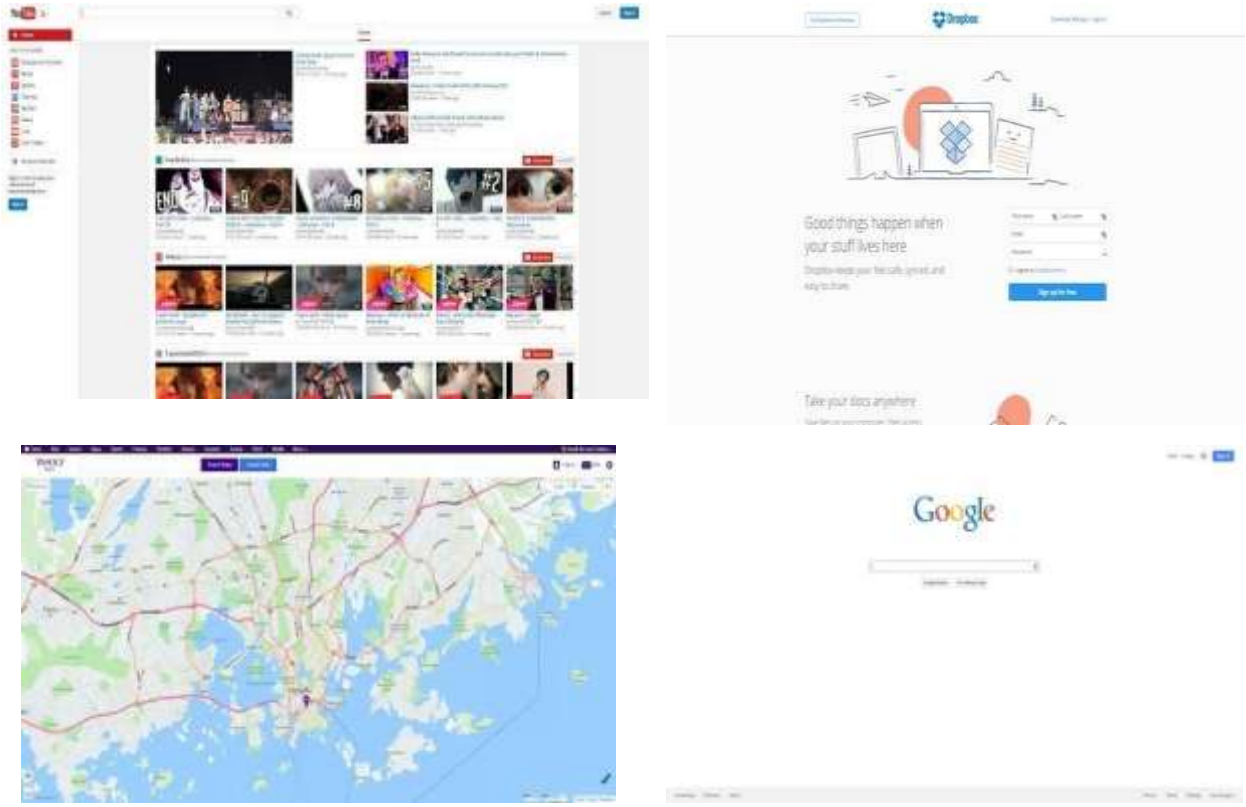
Theo thống kê từ Top 39 trường giảng dạy khoa học máy tính thì đa phần các trường sử dụng ngôn ngữ Python để giảng dạy:



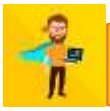
(Hình 4- Thống kế trường giảng dạy Python- nguồn viblo.asia)

Python được sử dụng chủ yếu trong các lĩnh vực:

- Lập trình web (*Youtube, Google, Instagram*)
- Phân tích dữ liệu, số liệu (*big data*)
- Xử lý toán học *(Pynum*)
- Tạo biểu đồ, hình hoá số liệu (*visualization*)
- Bảo mật mạng và máy tính
- Tự động hoá (*tự động crawl data chẳng hạn*)
- Scripting trong các engine (*Blender 3D, Maya Scripting...)*
- Lập trình robotics
- Tạo ứng dụng máy tính
- Scripting trong game (*nhiều game online dùng scripting bằng Python*)
- Xử lý đồ hoạ, ảnh kỹ thuật số.



(Hình 5. Một số ứng dụng nổi tiếng được viết bằng Python: Youtube, DropBox…)

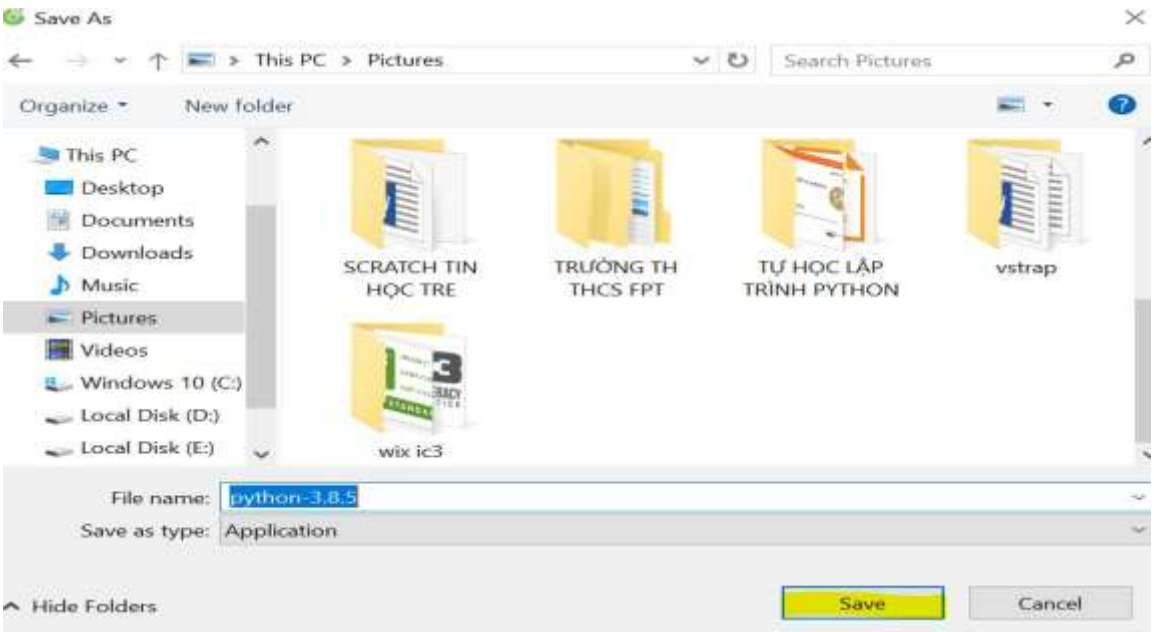


2. Cài đặt Python

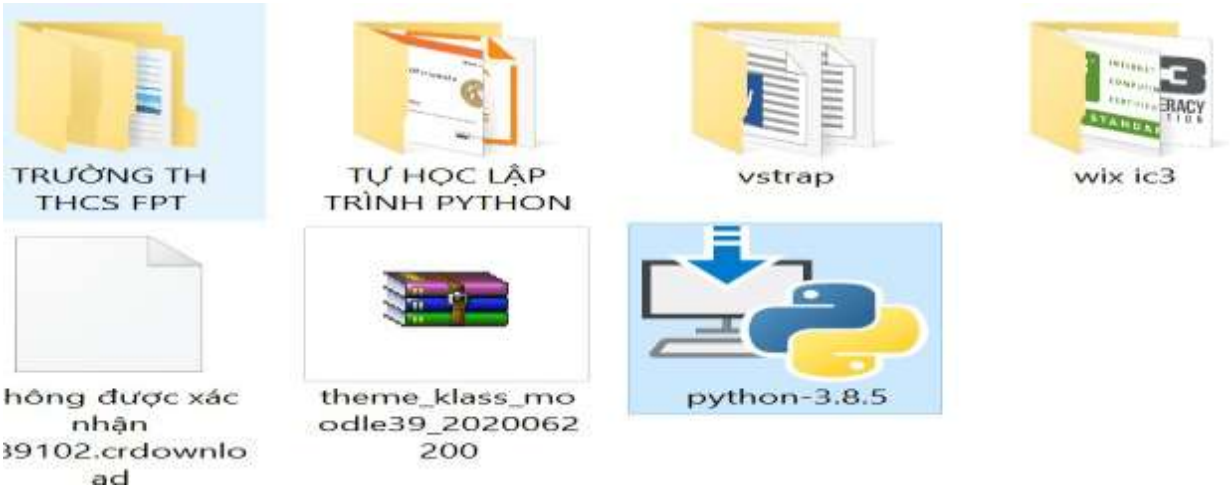
Bước 1. Để tải Python các em vào: https://www.python.org/downloads/



Bước 2. Ở màn hình download trên, các bạn chọn "Download Python 3" và một cửa sổ xuất hiện:



Các em chọn nơi lưu trữ python3.x rồi nhấn Save, chờ hệ thống tải về và kết quả:



Bước 3. Cài đặt.

Để cài đặt Python, các em nhấn đúp vào tập tin **python-3x.exe** vừa tải xong, nếu có hỏi gì về bảo mật thì nhấn Run:



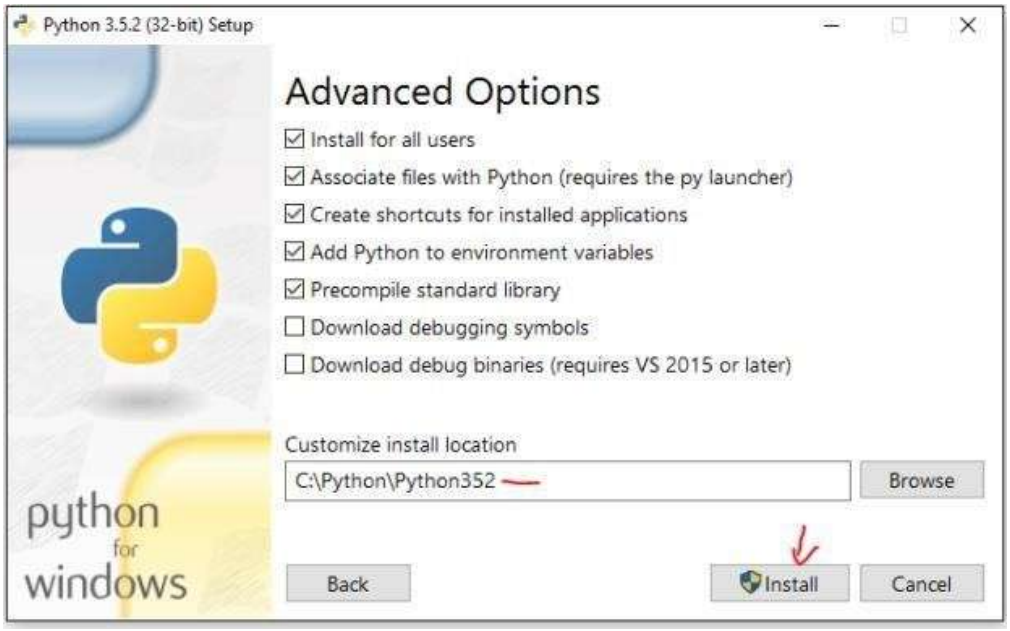
Sau đó màn hình chọn cấu hình để cài đặt sẽ hiển thị ra như dưới đây:



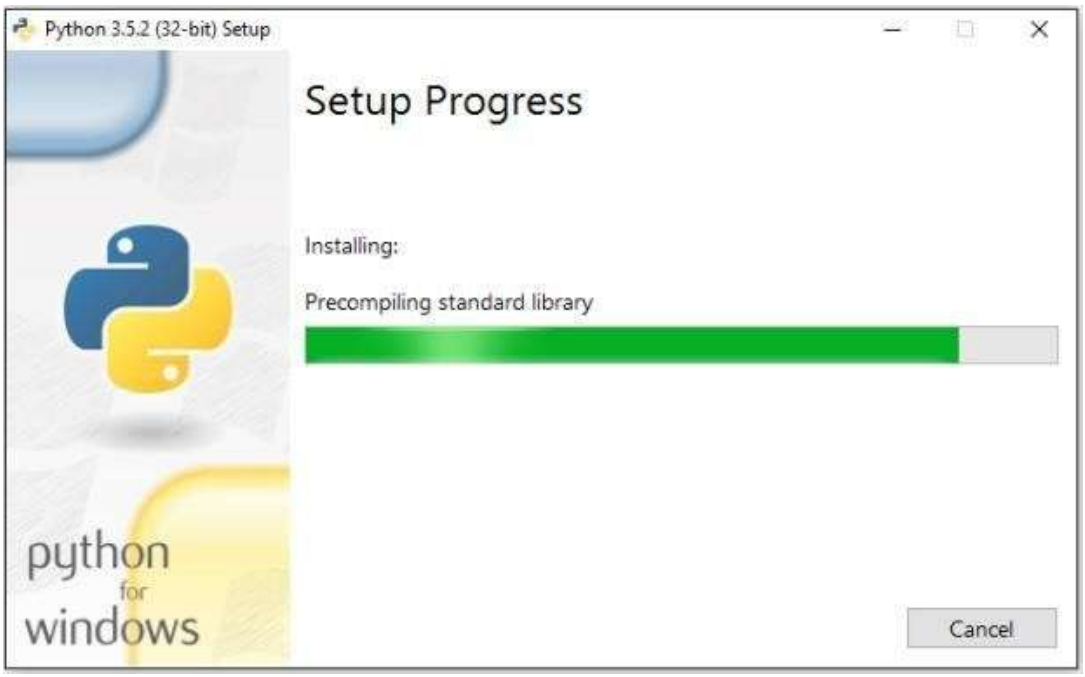
Các em chọn cấu hình như chỗ Thầy khoanh đỏ, rồi bấm Install Now hoặc *Customize Installation* (thông thường ta nên chọn *Customize installation* để chọn chỗ cài đặt khác cho đơn. Giờ Thầy chọn *Customize Installation*:



Các em chọn các cấu hình như trên rồi bấm Next:


Python 3.5.2 (32-bit) Setup
Advanced Options
Install for all users
Associate files with Python (requires the py launcher)
Create shortcuts for installed applications
Add Python to environment variables
Precompile standard library
Download debugging symbols
Download debug binaries (requires VS 2015 or later)
Customize install location
C:\Python\Python352
Browse
python
for
windows
Back
Install
Cancel

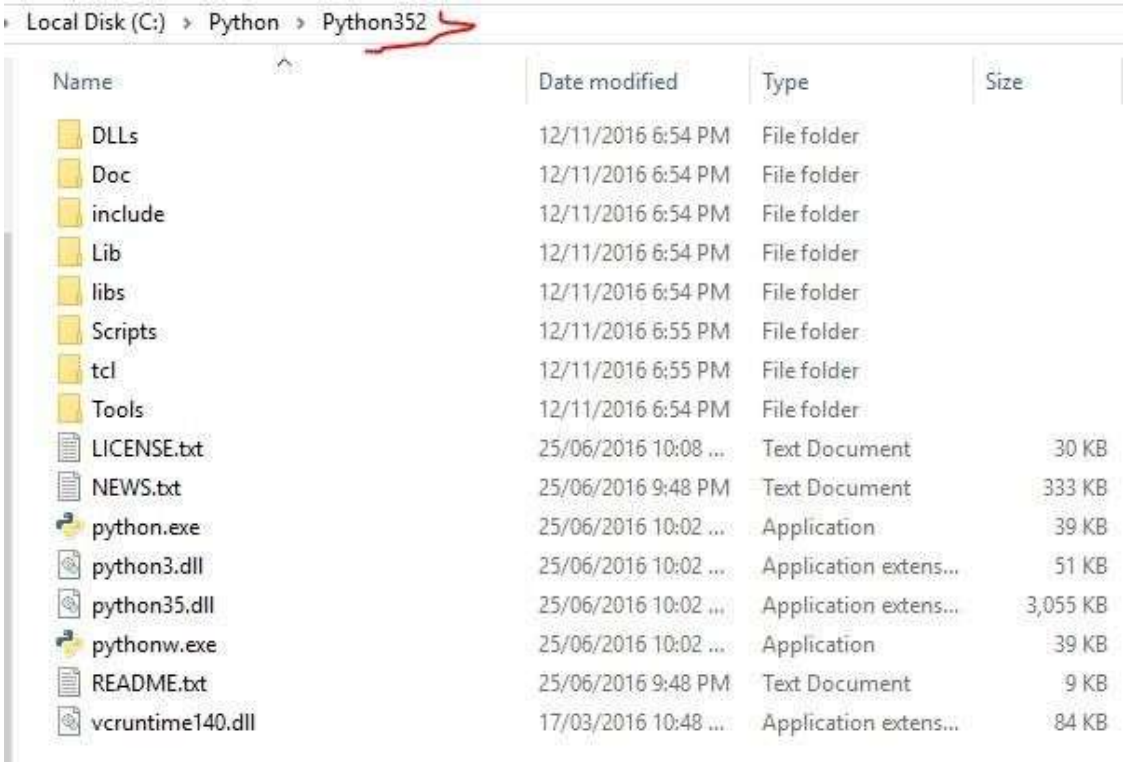
Ở hình trên các em chọn cấu hình giống vậy, chỗ nơi cài đặt chọn là ổ C, sau đó bấm Install:

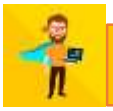


Các em chờ chương trình Python cài đặt (chỗ này chờ hơi lâu), khi nào báo cửa sổ bên dưới coi như là cài đặt thành công.



Các em kiểm tra ổ C ta có kết quả sau khi cài đặt:

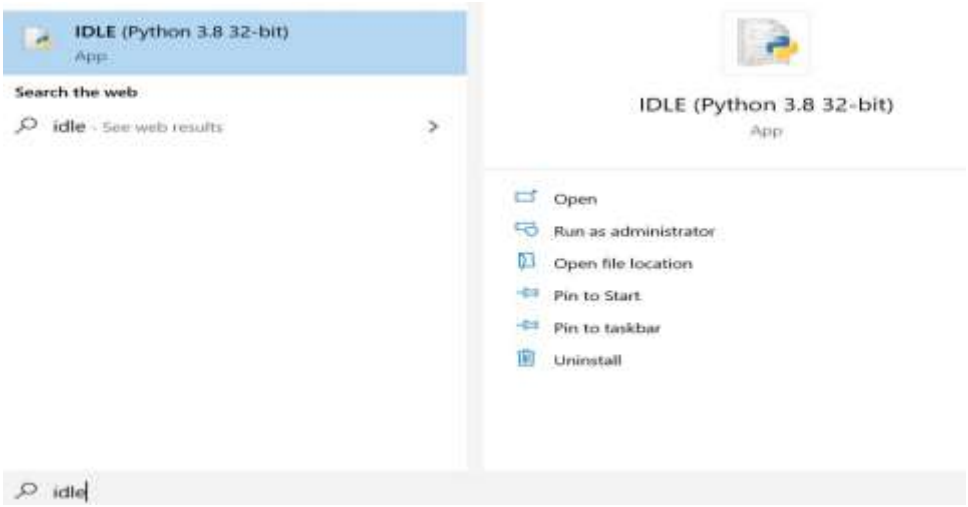




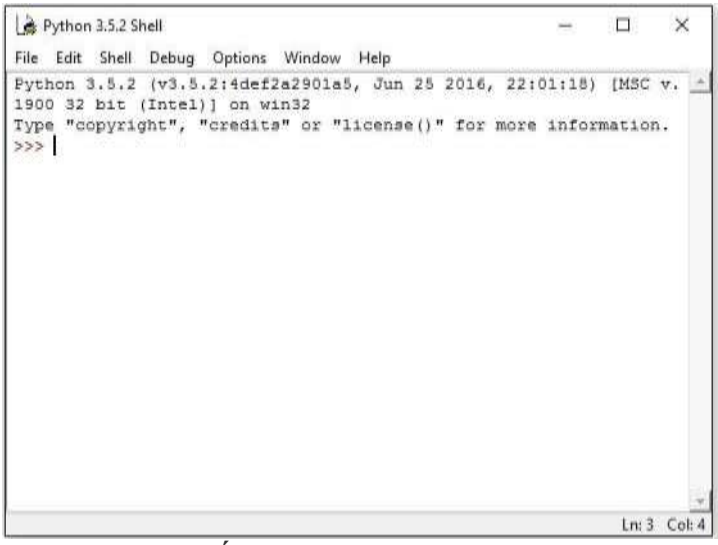
3. Một số công cụ lập trình

3.1 Công cụ lập trình chính thống sau khi cài Python

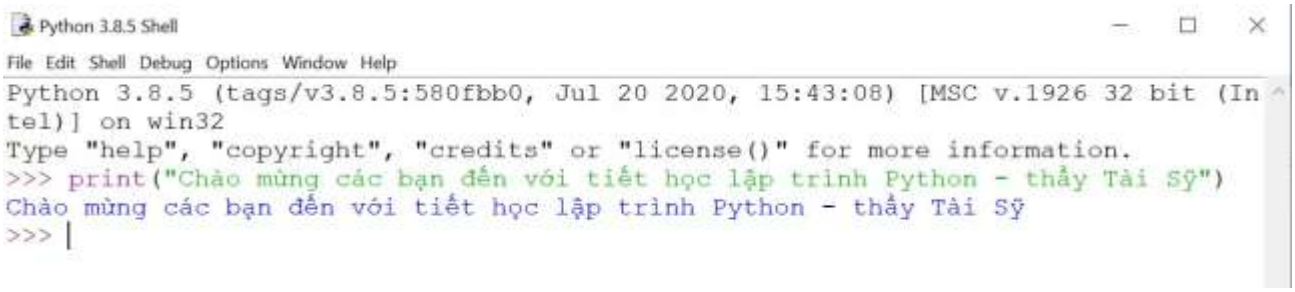
Đó chính là công cụ **IDLE** khi cài đặt xong. Tại ô Search ta gõ idle:



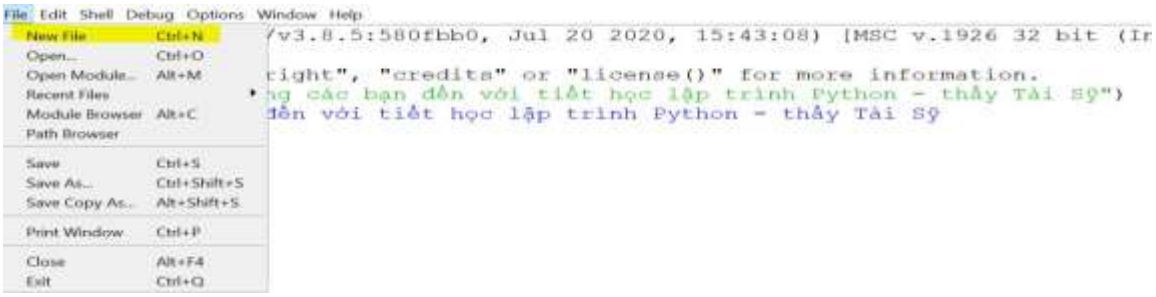
Các em nhấn chọn để khởi động phần mềm soạn thảo coding Python (nên đưa ra màn hình desktop nếu sử dụng nó thường xuyên), khi khởi động mặc định Python Power Shell sẽ xuất hiện như hình dưới đây:



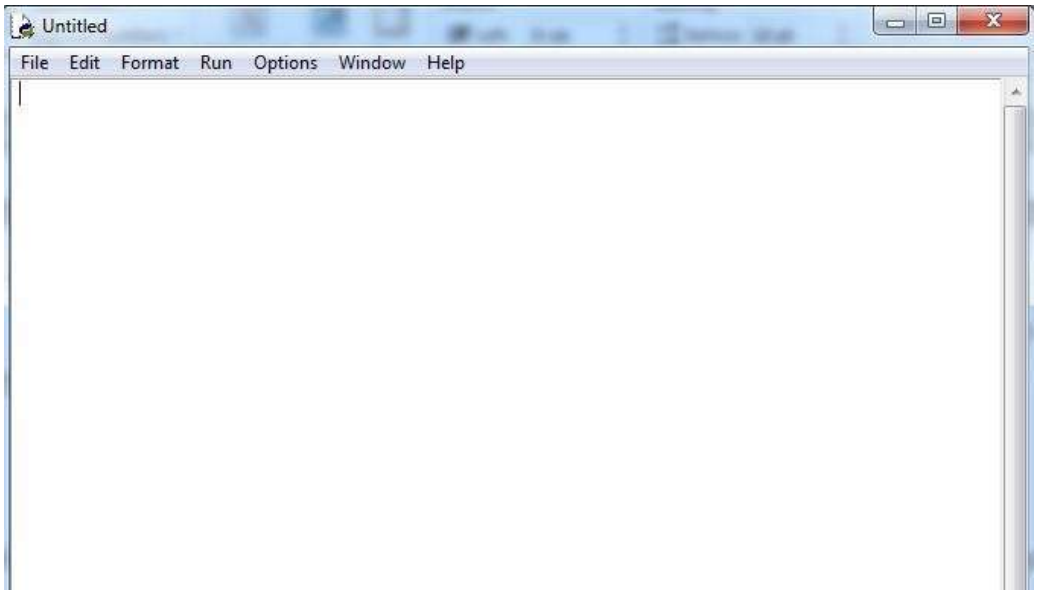
Ta có thể gõ các lệnh trực tiếp vào trong màn hình này, ví dụ:



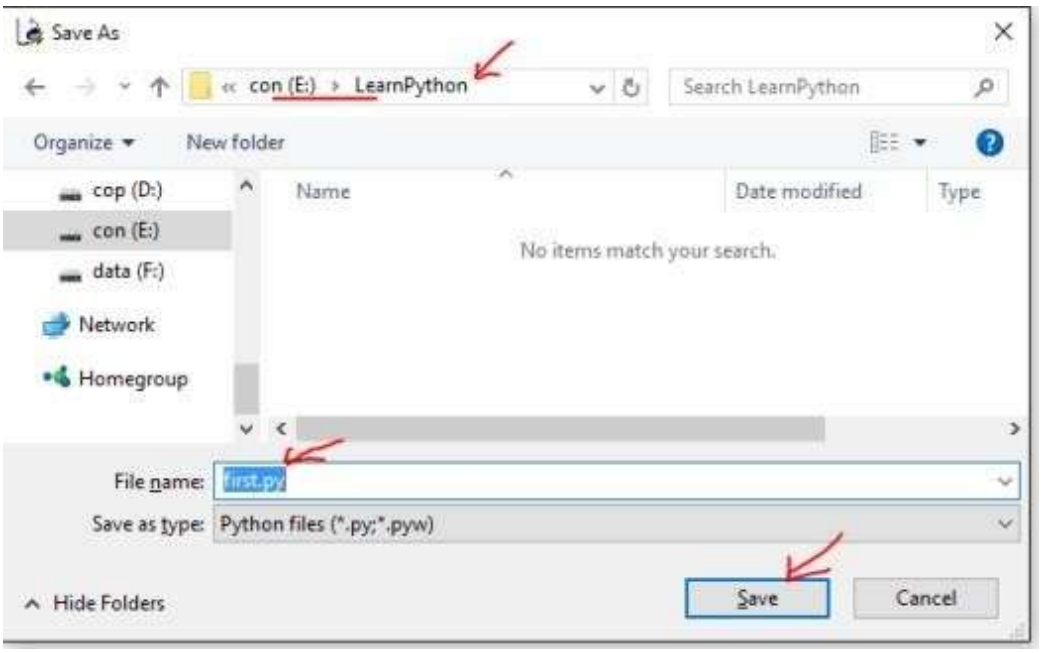
Ngoài ra ta cũng có thể tạo File để soạn thảo bằng cách vào File/ chọn New File, như hình họa hình bên dưới:



Sau khi nhấn New, màn hình soạn thảo hiện ra như dưới đây:

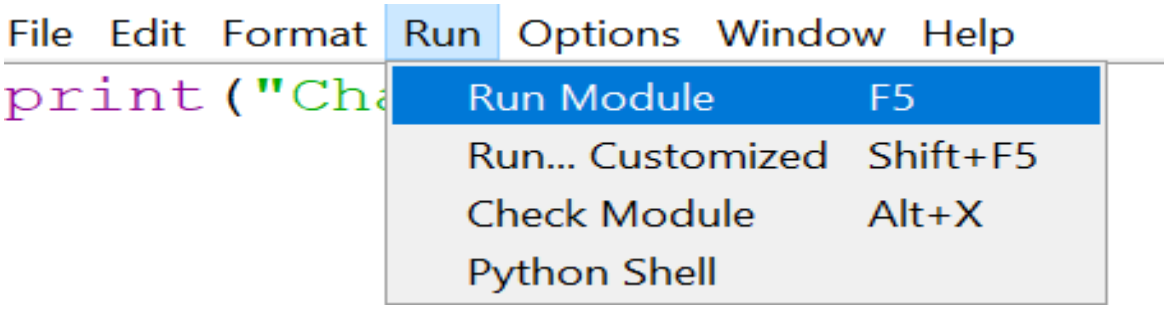


Trước khi chạy chương trình, Python yêu cầu chúng ta phải lưu lại bài làm. Các em vào File chọn Save, một cửa sổ hiển thị:



Các em chọn vị trí lưu và đặt tên file muốn lưu, ví dụ: first.py.

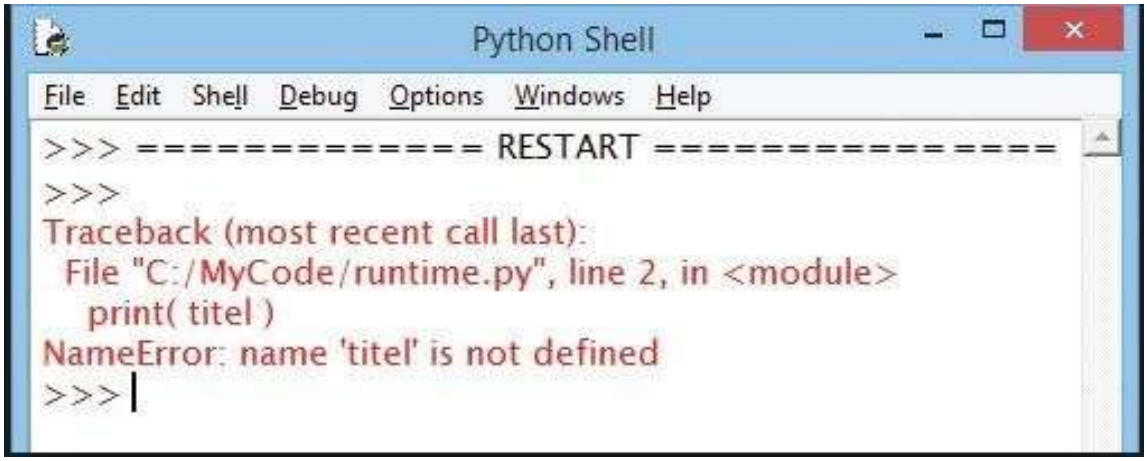
Để chạy, chương trình, các em nhấn F5 hoặc vào menu *Run/Run Module*



Chương trình cho ra kết quả:

```
File Edit Shell Debug Options Window Help
Python 3.8.5 (tags/v3.8.5:580fbb0, Jul 20 2020, 15:43:08) [MSC v.1926 32 bit (In
tel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/Admin/first.py ==========
Chào FPTer
>>>
```

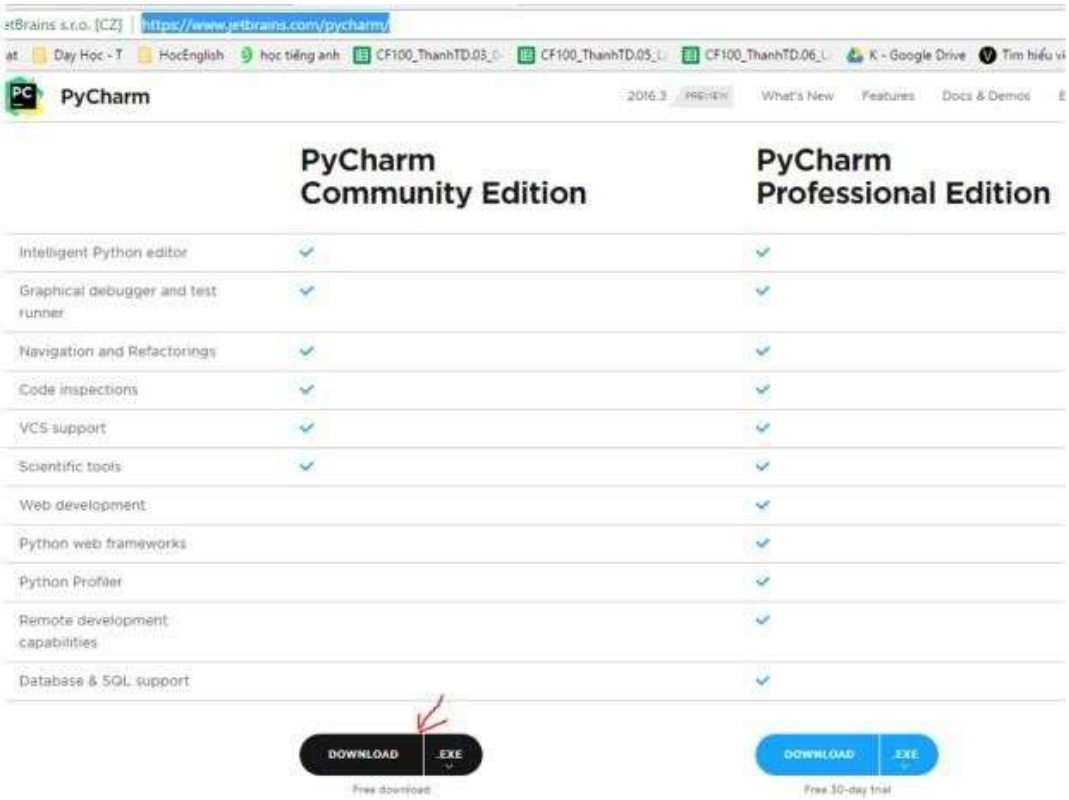
Trong quá trình biên dịch, nếu phần code của các em lỗi, thì Python sẽ báo lỗi, ví dụ:



2. **Công cụ lập trình PyCharm**

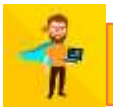
Đây là công cụ rất nổi tiếng, rất quen thuộc đặc biệt với những ai đã lập trình

Android Studio. Hiện nay, hãng cho sử dụng bản miễn phí PyCharm Community Edition: https://www.jetbrains.com/pycharm/ (các em kéo xuống gần dưới cùng sẽ có màn hình tải)



Các em tải về và tìm hiểu cách sử dụng nhé!





4. Giới thiệu hàm print

Hàm print(*nội dung muốn hiển thị***):** Dùng để hiển thị thông tin ra màn hình.

Nội dung muốn hiển thị nếu là ký tự, phải để trong dấu nháy kép hoặc nháy đơn; nếu *nội dung hiển thị* là số thì không cần đặt trong dấu nháy kép hoặc đơn.

Ví dụ:

```
File  Edit  Format  Run  Options  Window  Help
print("Trường FPT Cầu Giấy ")
print("Xin chào các bạn đến tiết học lập trình Python")
```

Ví dụ:

```
1  print("Trường FPT Cầu Giấy ")
2  print("Xin chào các bạn đến tiết học lập trình Python")
```

Kết quả khi chạy:

Trường FPT Cầu Giấy

Xin chào các bạn đến tiết học lập trình Python

Kết thúc mỗi dòng lệnh print, python sẽ tự động xuống dòng. Đôi khi chúng ta muốn các thông tin hiển thị trên cùng 1 dòng, chúng ta có thể làm điều này bằng cách thêm từ khóa end vào trong hàm print, ví dụ:

```
1    print(“FPT Schools",end=' ')
2    print("Xin chào",end=' ')
3    print("các bạn ")
```

Kết quả sẽ in ra: *FPT Schools xin chào các bạn*

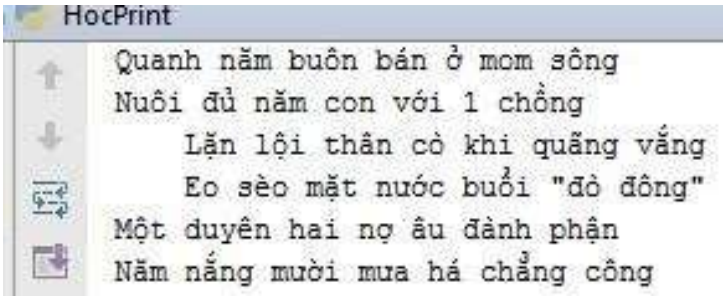
Ngoài ra Python còn hỗ trợ hiển thị một số ký tự đặc biệt như:

\\	In ra ký tự \
\”	In ra ký tự “
\’	In ra ký tự ‘
\t	In ra ký tự Tab
\n	In ra ký tự xuống dòng

Ví dụ 3:

```
1    print("Quanh năm buôn bán ở mom sông")
2    print("Nuôi đủ năm con với 1 chồng")
3    print("\tLặn lội thân cò khi quãng vắng")
4    print("\tEo sèo mặt nước buổi \"đò đông\"")
5    print("Một duyên hai nợ âu đành phận\nNăm nắng mười mưa há
     chẳng công")
```

Kết quả:



Ngoài ra hàm print còn cho phép chúng ta *lặp một chuỗi nào đó liên tục* với số lần tùy chọn, bằng cách dùng phép nhân *, ví dụ 4:

```
File Edit Format Run Options Window Help
print("*"*40)
print("Xin chào các bạn đến tiết học lập trình Python của thầy Sỹ")
print("*"*40)
```

Ta có kết quả khi chạy chương trình:

```
========== RESTART: C:/Users/Admin/first.py ==========
****************************************
Xin chào các bạn đến tiết học lập trình Python của thầy Sỹ
****************************************
>>>
```

Ở ví dụ trên, dòng lệnh print('*'*40): Cho chương trình hiển thị 40 lần dấu *



Trong bài số 1 này, chúng ta cần nhớ một số nội dung quan trọng sau:

- Lịch sử ra đời và phát triển của Python
- Các lĩnh vực ứng dụng Python

- Cách tải và cài đặt Python
- Sử dụng Python IDLE
- Sử dụng Pycharm
- Sử dụng hàm print() để hiển thị

Trong bài tiếp theo, Thầy sẽ giới thiệu cho các em về BIẾN, CÁC KIỂU DỮ LIỆU VÀ HÀM INPUT.

BÀI TẬP THỰC HÀNH

Bài 1. Các em hãy dùng hàm print() để in ra kết quả sau:

CHÀO MỪNG CÁC EM ĐẾN VỚI TIẾT HỌC ĐẦU TIÊN CỦA LẬP TRÌNH PYTHON

Em tên là gì:

Em học lớp? bao nhiêu tuổi?

Em hay giải bài toán sau:

23 + 25 + 22

80:4 -33

quả là:

160

Bài 2: Dùng lệnh print, in các hình sau:

```
*
***
*****
*******
*********
```

Bài 3: Dùng lệnh print, in các hình sau:

```
    *
   ***
  *****
 *******
*********
```

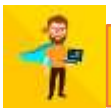
Bài 4: Dùng lệnh print, in các hình sau:

```
    *
   ***
  *****
 *******
    *
    *
    *
```

BÀI SỐ 2. BIẾN VÀ KIỂU DỮ LIỆU TRONG PYTHON

Nội dung bài học

- **Tìm hiểu về biến (variables)**
- **Tìm hiểu các kiểu dữ liệu**
- **Sử dụng toán tử toán học cơ bản trong Python**
- **Sử dụng các cách ghi chú lệnh trong Python**
- **Sử dụng hàm input()**
- **Vận dụng để giải quyết các bài toán thực tiễn**



1. Khái niệm, khai báo biến

1.1. Khái niệm Biến

Ví dụ: **X + 4 = 10; X – 83 = 26 x 7**

Trên đây là hình ảnh mà chúng ta thường xuyên gặp trong môn Toán và công việc của chúng ta là phải giải để tìm giá trị của "X". "X" là đại diện một số nào đó mà chúng ta chưa biết giá trị và ta cần tìm giá trị của nó. Trong lập trình các "X" như thế được gọi là biến.

Biến là các đối tượng sử dụng để chứa giá trị nào đó và giá trị đó có thể thay đổi trong suốt quá trình thực hiện chương trình.

Biến cũng vậy, nó là một đối tượng dùng để mang giá trị, giá trị của biến có thể thay đổi trong suốt quá trình sử dụng nếu cần thiết.

1.2. Khai báo biến trong Python

Để khai báo biến trong Python thì mọi người sử dụng cú pháp:

```
Ten_Bien = Gia_Tri
```

Trong đó:

- **Ten_Bien** là tên của biến mà các em muốn đặt. Tên biến này không được bắt đầu bằng số hay các ký tự đặc biệt, mà chỉ được bắt đầu bằng chữ cái hoặc ký tự _ và nó có phân biệt hoa thường.
- **Gia_Tri:** là giá trị của biến mà bạn muốn gán.

Ví dụ:

```
9fpt =10   # khai báo thế này, python sẽ báo lỗi vì có số ở đầu tên biến bc
fpt = 10   # khai đúng
```

Ngoài ra, các em cũng có thể khai báo nhiều biến bằng 1 giá trị trên 1 lần khai báo.

Ví dụ:

```
File Edit Format Run Options Window Help
a = b = c =10
```

Hay là khai báo nhiều biến với các giá trị tương ứng của nó trên 1 dòng.

Ví dụ:

```
File Edit Format Run Options Window Help
fullname, faculty = "Nguyễn Tài Sỹ", "IT"
```

1.3. Xóa biến trong Python

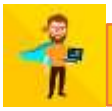
Trong Python có một điểm thú vị là: Nếu biến đó đang tồn tại mà ta xóa nó đi thì không còn sử dụng được nữa Python dùng từ khóa: **del** để xóa:

```
File  Edit  Format  Run  Options  Window  Help
x="Trường FPT Schools "
print(x)
del   x
print(x)
```

Lệnh trên chạy lên sẽ bị báo lỗi ngay dòng xuất x thứ 2, chi tiết kết quả:

```
========== RESTART: C:/Users/Admin/first.py =
Trường FPT Schools
Traceback (most recent call last):
  File "C:/Users/Admin/first.py", line 4, in <module>
    print(x)
NameError: name 'x' is not defined
```

Xuất hiện lỗi: *NameError: name 'x' is not defined* là do biến x đã bị xóa



2. Các kiểu dữ liệu trong Python

2.1. Kiểu dữ liệu

Các loại dữ liệu mà biến có thể chứa trong Python được mô tả ở bảng sau:

Kiểu dữ liệu	**Chú thích**
Kiểu **int**: Kiểu số nguyên	Kiểu số nguyên (không có chứa dấu chấm thập phân), có thể lưu các số nguyên âm và dương. *Ví dụ*: 223, -432
Kiểu **float**: Kiểu số thực	Kiểu số thực (có chứa dấu chấm thập phân), *Ví dụ*: 9.6, -10.4
Kiểu **str**: Kiểu chuỗi	Kiểu chuỗi, để trong nháy đôi hoặc nháy đơn *Ví dụ*: "FPT", 'FPT Cầu Giấy'

Kiểu **bool**: Kiểu luận lý	Kiểu luận lý, để lưu True hoặc False *Ví dụ 1*: t1=True *Ví dụ 2*: t2=False
Ngoài ra, còn 1 số kiểu khác nữa, nhưng chúng ta sẽ tập trung vào các kiểu dữ liệu cơ bản.	

Trong Python một biến không cần khai báo kiểu dữ liệu, khi ta gán giá trị thì tự động Python **sẽ tự ra kiểu dữ liệu** của biến. Như vậy một biến có thể có nhiều kiểu dữ liệu **tùy thuộc vào giá trị mà ta gán**. Ta có thể dùng hàm **type**() để kiểm tra kiểu dữ liệu của biến

Ví dụ:

File Edit Format Run Options Window Help

```
x=5
print(x,type(x))
x=5.5
print(x,type(x))
x='teo'
print(x,type(x))
x=True
print(x,type(x))
```

Kết quả lần lượt sẽ có kiểu dữ liệu của x là:

```
========== RESTART: C:/Users/Admin/first.py
5 <class 'int'>
5.5 <class 'float'>
teo <class 'str'>
True <class 'bool'>
```

2.2. Ép kiểu dữ liệu trong Python

Trong một trường hợp nào đó mà các em muốn chuyển đổi kiểu dữ liệu của một biến, thì Python cũng hỗ trợ qua các hàm cơ bản sau:

float(data) chuyển đổi sang kiểu số thực.

int(data,base) chuyển đổi sang kiểu số, trong đó base là kiểu hệ số mà các bạn muốn chuyển đổi sang (tham số này có thể bỏ trống).

str(data) chuyển đổi sang dạng chuỗi.

Ví dụ:

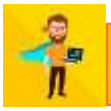
```
age = 22;

# ép sang float
floatAge = float(age)
print(type(floatAge))

#ép sang integer.
intAge = int(age)
print(type(intAge))

#ép sang chuỗi.
strAge = str(age)
print(type(strAge))
```

Trong ví dụ trên, các em thấy xuất hiện một số dòng lệnh như: *# ép sang float* hay *#ép sang integer*… đây là cách ghi chú lệnh thầy sẽ trình bày ở phần 4.



3. Các toán tử trong Python

Mỗi một ngôn ngữ lập trình đều có tập các toán tử thường dùng và đa phần chúng khá giống nhau. Những bạn nào đã học C++, java, C# thì qua Python cũng tương tự. Các toán tử cơ bản

1. Toán tử số học cơ bản

2. Toán tử gán

3. Toán tử so sánh

4. Toán tử logic

Trong bài này, thầy sẽ giới thiệu cho các em về toán tử số học cơ bản và gán, các toán tử tiếp theo **thầy sẽ giới thiệu ở các bài sau.**

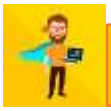
3.1. Toán tử số học cơ bản:

Toán tử	Mô tả	Ví dụ
+	Cộng	12 + 4.9 => kết quả 16.9
–	Trừ	3.98 – 4 => kết quả -0.02
*	Nhân	2 * 3.4 => kết quả 6.8
/	Chia	9 / 2 => kết quả 4.5
//	Chia lấy phần nguyên	9 // 2 => kết quả 4
%	Chia lấy phần dư	9%2 =>kết quả 1
**	Lũy thừa	3**4=>kết quả 81

3.2. Toán tử gán:

Toán tử	Mô tả	Ví dụ	Tương đương với
=	Phép gán giá trị bên phải cho biến bên trái dấu bằng	x=5	
+=	Cộng và gán	x=2 x+=5 ==>x=7	x=x+5
-=	Trừ và gán	x=2 x-=5 ==>x=-3	x=x-5

*=	Nhân và gán	x=2 x*=5 ==>x=10	x=x*5
/=	Chia và gán	x=7 x/=5 ==>x=1.4	x=x/5
//=	Chia và gán (lấy nguyên)	x=7 x//=5 ==>x=1	x=x//5
%=	Chia lấy dư	x=7 x%=5 ==>x=2	x=x%5
**=	Lấy lũy thừa và gán	x=2x**=3 ==>x là 2 mũ 3 =8	x=x**3



4. Ghi chú trong Python

Ví dụ:



4.2. Ghi chú nhiều dòng: Dùng **“””** *nội dung cần ghi chú* **“””** **(***3 cặp nháy đôi)* **hoặc** **’’’** *nội dung cần ghi chú* **’’’** **(*3 cặp nháy đơn*)**

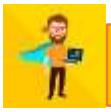
Ví dụ dùng 3 cặp nháy đôi:

```
1   """
2   Trường TH & THCS FPT Cầu Giấy:
3   – Học và luyện thi chứng chỉ IC3
```

```
4       – Học lập trình Scratch, Python, HtML
5       – CLB Siêu nhân lập trình Python, CLB Creating 3D VR
6       """
          print("Xin chào các FPTer")
```

Ví dụ dùng 3 cặp nháy đơn:

```
6       '''
7       Trường TH & THCS FPT Cầu Giấy:
8       – Học và luyện thi chứng chỉ IC3
9       – Học lập trình Scratch, Python, HtML
10      – CLB Siêu nhân lập trình Python, CLB Creating 3D VR
11      '''
          print("Xin chào các FPTer")
```



5. Giới thiệu hàm input() trong Python

Trong Python để nhập liệu từ bàn phím ta dùng hàm input(). Giá trị nhập vào của hàm input() thường là kiểu chuỗi, do đó ta cần chuyển kiểu nếu như muốn lưu trữ giá trị nhập vào không phải kiểu chuỗi.

Bây giờ ta tạo một tập tin tên là: **nhaplieu.py** với các lệnh sau:

*nhaplieu.py - C:/Users/Admin/Pictures/TRƯỜNG TH THCS FPT/CLB LẬP TRÌNH PYTHON/Coding python/nhaplieu.py

File Edit Format Run Options Window Help

```
# Hàm input()
print("Em hãy nhập cái gì đó từ bàn phím:")
a=input()
print("Bạn nhập:",a)
print("Kiểu dữ liệu:",type(a))
```

Các em chạy File: nhaplieu.py, sau đó nhập một vài dữ liệu để kiểm tra:

Trường hợp 1: Bạn nhập giá trị là 100, ta có kết quả:

```
Em hãy nhập cái gì đó từ bàn phím:
100
Bạn nhập: 100
Kiểu dữ liệu: <class 'str'>
```

Các em quan sát ta nhập 100, nhưng kiểu dữ liệu vẫn là chuỗi str

Trường hợp 2: Nhập giá trị là 10.12, ta có kết quả:

```
Em hãy nhập cái gì đó từ bàn phím:
10,12
Bạn nhập: 10,12
Kiểu dữ liệu: <class 'str'>
```

Các em quan sát ta nhập 10.12, nhưng kiểu dữ liệu vẫn là chuỗi str

Trường hợp 3: Nhập giá trị là FPTER, ta có kết quả:

```
Em hãy nhập cái gì đó từ bàn phím:
FPTER
Bạn nhập: FPTER
Kiểu dữ liệu: <class 'str'>
```

Trường hợp 4: Nhập giá trị là True, ta có kết quả:

```
Em hãy nhập cái gì đó từ bàn phím:
True
Bạn nhập: True
Kiểu dữ liệu: <class 'str'>
```

Tóm lại khi sử dụng hàm input() thì mọi trường hợp điều là kiểu chuỗi str (cho dù các em có nhập loại dữ liệu nào đi nữa)

Ta cần ép kiểu dữ liệu

Ví dụ: Lấy giá trị nhập vào là **kiểu int:**

```
# Hàm input(), Ép kiểu dữ liệu
print("Mời Em nhập int:")
x=int(input())
print("Bạn nhập:",x)
print("Kiểu dữ liệu:",type(x))
```

Bạn nhập giá trị là 100, Ta có kết quả:

```
Mời Em nhập int:
100
Bạn nhập: 100
Kiểu dữ liệu: <class 'int'>
```

Như vậy ta có thể dùng int() để ép kiểu chuỗi về kiểu int.

Tương tự bạn có thể **ép về kiểu số thực float:**

```
# Hàm input(), Ép kiểu dữ liệu
print("Mời Em nhập float:")
x=float(input())
print("Bạn nhập:",x)
print("Kiểu dữ liệu:",type(x))
```

Ta có kết quả:

```
Mời Em nhập float:
10.12
Bạn nhập: 10.12
Kiểu dữ liệu: <class 'float'>
```

Ép kiểu True, False từ str sang bool

```
# Hàm input(), Ép kiểu dữ liệu
x=bool(input("Mời Em nhập True hoặc False: "))
print("Bạn nhập:",x)
print("Kiểu dữ liệu:",type(x))
```

Ta có kết quả:

```
Mời Em nhập True hoặc False: True
Bạn nhập: True
Kiểu dữ liệu: <class 'bool'>
```

Tương tự ta có thể ép kiểu từ int, float sang str



Xin chúc mừng các em đã hoàn thành xong bài số 2, trong này chúng ta cần nhớ một số nội dung quan trọng sau:

BIẾN:

1. Biến là các đối tượng sử dụng để chứa giá trị nào đó và, giá trị đó có thể thay đổi trong suốt quá trình thực hiện chương trình
2. Khai báo biến: TenBien= GiatriBien
3. Để xóa biến dùng hàm: **del()** để xóa **KIỂU DỮ LIỆU:**

1. Các loại dữ liệu mà biến có thể chứa trong Python: số nguyên (int), số thực(float), chuỗi(str), số phức **(**complex**),** kiểu luận lý (bool).
2. Dùng hàm **type**() để kiểm tra kiểu dữ liệu của biến
3. Có thể ép kiểu dữ liệu: *float(data), int(data,base), str(data),…*

TOÁN TỬ SỐ HỌC VÀ TOÁN TỬ GÁN:

1. Toán tử số học cơ bản bao gồm: +, -, *, /, %, **
2. Toán tử gán: =, +=, -=, *=, /=, //=, %=, **=

GHI CHÚ LỆNH

1. Ghi chú 1 dòng: Dùng từ khóa #
2. Ghi chú nhiều dòng: Dùng “”” nội dung cần ghi chú “”” (3 cặp nháy đôi) hoặc ”’ nội dung cần ghi chú ”' (3 cặp nháy đơn)

HÀM INPUT()

Hàm input() thường là kiểu chuỗi, do đó ta cần chuyển kiểu nếu như muốn lưu trữ giá trị nhập vào không phải kiểu chuỗi

BÀI TẬP THỰC H ÀNH

Bài 1. Các dòng mã sau đây có in cùng một thứ không? Giải thích lý do tại sao và tại sao không.

print(100)

print("100")

Bài 2. Các dòng mã sau đây có in cùng một thứ không? Giải thích lý do tại sao và tại sao không.

x = 10

print(x)

print("x")

Bài 3. Điều gì sai với câu lệnh cố gán giá trị 10 cho biến x?

10 = x

Bài 4. Sự khác biệt giữa hai chuỗi sau đây là gì? 'n' và '\n'**?**

Bài 5. Sự khác nhau giữa hai biến a và b dưới đây là gì?

a = 0

b = 0.0

Bài 6. Hãy viết một chương trình nhập tên và tuổi của các em vào; máy tính sẽ tính số năm để các em đạt 100 tuổi.

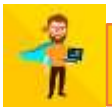
Gợi ý:

```
name = input("Enter your name: ")
print("My name is: ", name)
age = int(input("Enter your age: "))
print("I'm : ", age," years old")
print(100-age, " years from now, I'll be 100 years old.")
```

BÀI SỐ 3. CẤU TRÚC ĐIỀU KIỆN IF

Nội dung bài học:

- **Tìm hiểu về toán tử quan hệ và logic trong Python**
- **Nắm được cách sử dụng cấu trúc điều kiện if**
- **Nắm được cách sử dụng cấu trúc điều kiện if-else**
- **Nắm được cách sử dụng cấu trúc điều kiện if-elif-else**



1. Toán tử quan hệ và logic trong Python

Trong bài 2, chúng ta đã được tìm hiểu về các kiểu dữ liệu trong Python. Trong đó có kiểu dữ liệu Bool, kiểu dữ liệu này chỉ có hai giá trị: **True** và **False.** Trong kiểu dữ liệu Bool có sử dụng các toán tử quan hệ và toán tử logic cụ thể như sau:

```
>>> 10 > 5
True
>>> 10<= 5
False
>>> 10 >=5
True
>>> 10 != 5
True
>>> 10 == 5
False
```

1.1. Toán tử quan hệ:

Toán tử	Miêu tả
<	Nhỏ hơn.
>	Lớn hơn
<=	Nhỏ hơn hoặc bằng
>=	Lớn hơn hoặc bằng
==	Bằng

!=	Không bằng
<>	Không bằng (tương tự !=)

Ví dụ:

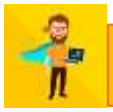
1.2. Toán tử logic trong Python

Python hỗ trợ các toán tử logic sau:

Toán tử	Miêu tả
and	Phép Và. Nếu cả hai điều kiện là true thì kết quả sẽ là true
or	Phép Hoặc. Nếu một trong hai điều kiện là true thì kết quả là true
not	Phép phủ định. Được sử dụng để đảo ngược trạng thái logic của toán hạng

Dưới đây là ví dụ minh họa cho các toán tử logic trong Python:

```
>>> 10 > 5 and 6 > 7
False
>>> 10 > 5 and 7 >= 6
True
>>> 10 > 5 or 6 > 7
True
>>> 10 > 15 and 6 > 7
False
>>> not 6 >7
True
```



2. Cấu trúc điều kiện if... và if...else

Trong cuộc sống thực tế chúng ta gặp rất nhiều trường hợp về xử lý tình huống theo điều kiện (còn gọi là Nếu ... Thì ...). Ví dụ như: Nếu tôi giàu tôi sẽ đi du lịch vòng quanh

thế giới. Hoặc đơn giản hơn giả sử bạn mua một vé xổ số bạn sẽ có cơ hội trúng giải đặc biệt, giải nhất, giải nhì... hoặc không trúng giải nào.

Trong Python cung cấp cấu trúc điều kiện if giúp chúng ta giải quyết những vấn đề trên.

2.1. Cấu trúc lệnh điều kiện if (nếu…. thì)

Cú pháp:

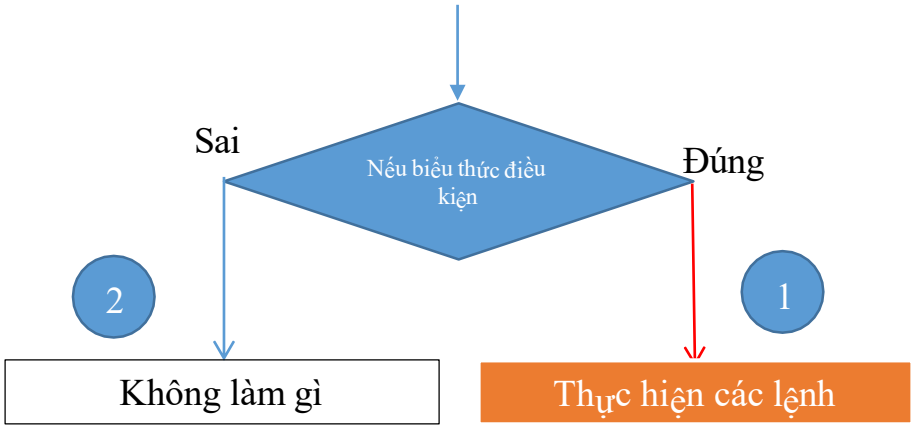
```
if Biểu_thức:

        Các câu lệnh
```

Trong đó: ***Biểu_thức:*** có sử dụng các toán tử so sánh, kết quả trả về: True hoặc False. Ví dụ: 3>5 kết quả trả về là False; 3==3 kết quả trả về là: true

Toán tử	Miêu tả
<	Nhỏ hơn.
>	Lớn hơn
<=	Nhỏ hơn hoặc bằng
<=	Lớn hơn hoặc bằng
==	Bằng
!=	Không bằng
<>	Không bằng (tương tự !=)

Để các em hiểu rõ về câu lệnh điều kiện if, ta có sơ đồ sau:



Khi python gặp câu lệnh if, chương trình sẽ kiểm tra biểu thức điều kiện, nếu biểu thức điều kiện là True thì (1) sẽ thực hiện, còn nếu False thì không làm gì cả.

Ví dụ:

```
num = int(input("Enter number: "))
if num > 10:
    print("That number is greater than 10")
```

Kết quả:

```
Enter number: 11
That number is greater than 10
```

Trở lại ví dụ điều kiện: **NẾU** *num thi > 10* **THÌ** *số đó lớn hơn 10.* Bây giờ thầy lại có câu điều kiện sau:

NẾU num > 10 **THÌ** *số đó lớn hơn 10* **NGƯỢC LẠI** *số đó nhỏ hơn hoặc bằng 10*

Trong Python cung cấp lệnh: if …. else… giúp chúng ta thực hiện những công việc trong trường hợp biểu thức điều kiện là False.

2.2. Cấu trúc lệnh điều kiện if...else

Cú pháp:

if điều_kiện:

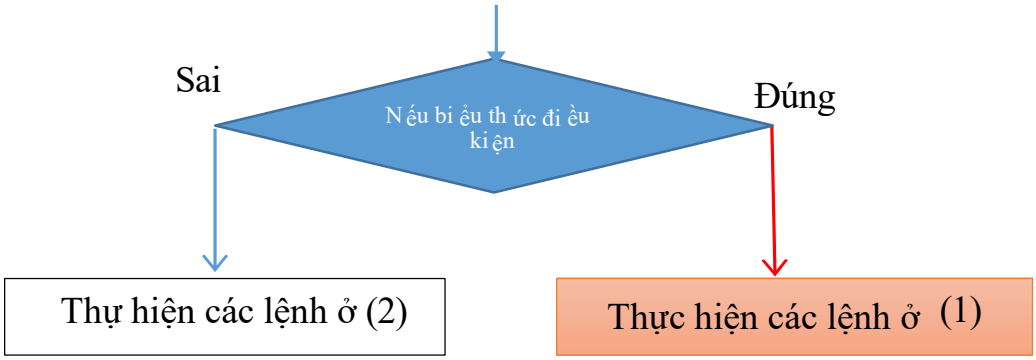
Các câu lệnh (1)

else:

Các câu lệnh (2)

Với câu lệnh trên: máy tính trước tiên sẽ kiểm tra **biểu thực điều kiện**, nếu *biểu thức điều kiện đúng* thì **(1)** sẽ thực hiện, nếu *biểu thức điều kiện sai* (không đúng) thì **(2)** sẽ thực hiên. **Chú ý: chỉ có duy nhất một lệnh else sau lệnh if.**

Sơ đồ if… else…

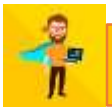


Ví dụ:

```
num = int(input("Enter number: "))
if num > 10:
    print("That number is greater than 10")
else:
    print("That number is less than or equal to than 10")
```

Kết quả:

```
Enter number: 5
That number is less than or equal to than 10
```



3. Lồng các lệnh if

Thầy có ví dụ sau: Sau khi thi kết thúc học kỳ với 3 môn thi: Toán, Văn, T.Anh.

Căn cứ vào điểm trung bình (ĐTB) của 3 môn: (Toán + Văn + Anh)/3 thầy sẽ đánh giá học sinh đó đạt thành tích học tập: xuất sắc, giỏi, khá hoặc trung bình, cụ thể như sau:

NẾU *ĐTB >=9* THÌ: *"Đạt loại xuất sắc"*

NGƯỢC LẠI:

NẾU *ĐTB>=8* THÌ: *"Đạt loại giỏi"*

NGƯỢC LẠI:

NẾU ĐTB >=7 THÌ: *"Đạt loại khá"*

NGƯỢC LẠI: "*đạt loại trung bình"*

Trong Python cung cấp cú pháp lệnh if lồng nhau để giúp ta thực hiện ví dụ trên:

- **Cú pháp**

```
if bieu_thuc1:
        cac_lenh (1)
elif bieu_thuc2:
        cac_lenh (2)
elif bieu_thuc3:
        cac_lenh (3)
else:
        cac_lenh (4)
```

Không giống else mà chỉ có một lệnh được theo sau if, các em có thể sử dụng **nhiều elif theo sau if.**

- **Ví dụ:** Giải quyết bài toán trên

```
# Nhập vào điểm của 3 môn thi:
print("Điểm toan: ")
_toan=input()
print("Nhập điểm Văn:")
_van=input()
print("Nhap điểm T.ANh")
_anh=input()

# TỔNG CỦA 3 ĐIỂM
_DTB=(int(_toan) + int(_van) + int(_anh))/3
print(_DTB)

# KIỂM TRA ĐẠT LOẠI GÌ:

if _DTB>=9:
    print("Em đạt loại xuất sắc")
elif _DTB>=8:
        print("Em đạt loại Giỏi")
elif _DTB>=7:
        print("em đạt loại Khá")
else:
    print("Em đạt loại Trung ")

```

Kết quả:

```
Điểm toan:
5
Nhập điểm Văn:
6
Nhap điểm T.ANh
7
6.0
Em đạt loại Trung
```



Xin chúc mừng các em đã hoàn thành xong bài số 3, trong này chúng ta cần nhớ một số nội dung quan trọng sau:

1. Biểu thức Boolean đại diện cho các giá trị True và False
2. Cái tên Boolean Được đặt tên theo nhà toán học George Boole, từ Boolean luôn bắt đầu bằng chữ viết hoa B. Các giá trị True và False cũng sẽ luôn có giá trị T và F tương ứng, vì chúng là các giá trị đặc biệt trong Python.
3. Các số khác không và các chuỗi khác rỗng biểu thị các giá trị True. Số 0 (số nguyên hoặc dấu phẩy động) và chuỗi rỗng biểu thị giá trị False trong biểu thức Boolean.
4. Toán tử quan hệ: ==; !=; <; >; <=; >=
5. Toán tử logic: and, or và not
6. Câu lệnh điều kiện: if; if…else… và if …. elif…. else

BÀI TẬP

Bài 1. Cho các khai báo sau:

x, y, z = 4, 6, 7

Các em hãy cho biết kết quả trả về của các biểu thức sau:

(a) x == 4

(b) x < y

(c) x >= y

(d) x <= y

(e) x != y - 2

(f) x < 10

(g) x >= 0 and x < 1

(h) x < 0 and x < 10

(i) x >= 0 and x < 2

(j) x < 0 or x < 10

(k) x > 0 or x < 10

(l) x < 0 or x > 10

Bài 2. Cho các khai báo sau:

b1, b2, b3, b4 = True, False, 3, 1

Các em hãy cho biết kết quả trả về của các biểu thức sau:

(a) b3

(b) b4

(c) not b1

(d) not b2

(e) not b3

(f) not b4

(g) b1 and b2

(h) b1 or b2

(i) b1 and b3 > 5

(j) b1 or b3 <3

(k) b1 and b4 > 5

(l) b1 or b4 > 5

(m) b2 and b3 < 0

(n) b2 or b3 >7

(o) b1 and b2 or b3 > 7

Bài 3. Viết chương trình Python yêu cầu giá trị nguyên nhập từ người dùng. Nếu giá trị nằm trong khoảng từ 1 đến 100, hãy in ra OK; nếu không, viết ra: "*số không nằm trong khoảng 1 – 100*"

Bài 4. Viết chương trình Python cho phép người dùng nhập vào một năm. Kiểm tra xem năm đó có phải năm nhuận hay không?

Gợi ý:

Năm nhuận là:

- Năm **chia hết cho 4** nhưng **không chia hết cho 100**.
- Hoặc năm **chia hết cho 400**.

Bài 5. Hãy xem xét đoạn mã Python sau:

```
# i, j, and k are numbers
if  i < j:
    if j < k:
        i = j
    else:
        j = k
else:
    if j > k:
        j = i
    else:
        i = k
print("i =", i, " j =", j, " k =", k)
```

Code trên sẽ in gì nếu giá trị của các biên I, j và k lần lượt là:

(a) i = 3, j = 5, k = 7

(b) i =3, j = 7, k = 5

(c) i = 7, j = 3, k = 5

Câu 6. Viết chương trình Python yêu cầu 5 giá trị nguyên từ người dùng. Sau đó, in ra giá trị lớn nhất và giá trị nhỏ nhất trong 5 số đó.

Bài 8. Hãy viết chương trình để hiển thị ra menu:

Các chương trình trong game show :
****
* 1. Cầu hỏi cơ bản
* 2. Ai thông minh hơn học sinh lớp 5
* 3. Ai là triệu phú
********** Bạn lựa chọn khóa học nào? (*nhấn 1 hoặc 2, 3*):

Sau khi chọn khóa học: chương trình hiển thị: (Ví dụ nhấn phím 2)

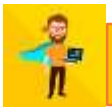
Chương trình xin cảm ơn bạn!
Bạn vừa chọn gói câu hỏi: 2. Ai thông minh hơn học sinh lớp 5

Chúc bạn may mắn !!!

BÀI SỐ 4. VÒNG LẶP WHILE

Nội dung bài học:

- **Sử dụng vòng lặp While trong Python - Nắm được câu lệnh break trong while**
- **Nắm được câu lệnh continue trong while**
- **Ứng dụng vòng lặp while trong các ví dụ thực tiễn.**



1. Giới thiệu vòng lặp While

Chúng ta hãy xét ví dụ sau:

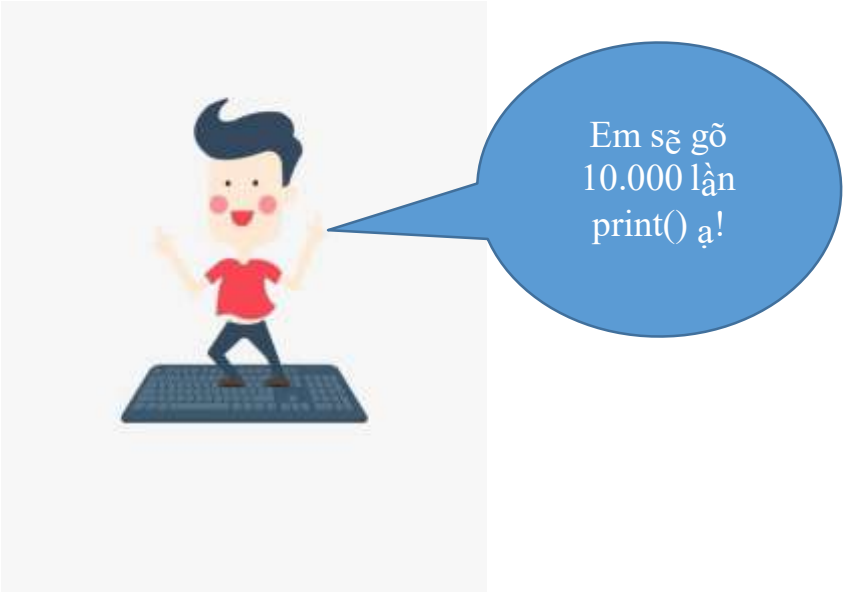
Ví dụ 4.1.

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Kết quả:



Vậy, sẽ ra sao nếu thầy muốn in ra từ 1 đến 10.000?





Các em sẽ gõ 10.000 lần???? như vậy quá mất thời gian. Trong Python cung cấp 1 câu lệnh giúp chúng ta giải quyết vấn đề trên. Đó là "*vòng lặp*" while.



Hiểu 1 cách đơn giản "vòng lặp" là thực hiện lặp đi lặp lại 1 công việc nào đó.

Ví dụ 4.2: *Ngày nào em cần đánh răng, rửa mặt mỗi khi thức dậy!*

Trong buổi học ngày hôm nay, thầy sẽ giới thiệu cho các em về vòng lặp While.

Cú pháp vòng lặp while:

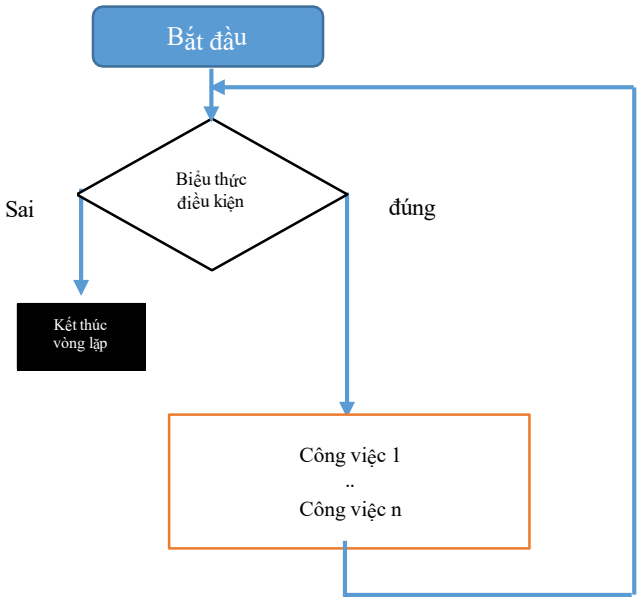
while (*biểu thức điều kiện***):**

Công việc 1

Công việc 2

Công việc n

Sơ đồ thực hiện lệnh while như sau:



Khi gặp câu lệnh while, chương trình sẽ thực hiện:

Bước 1. Kiểm tra biểu thức điều kiện (BTĐK)

Bước 2. + Nếu BTĐK là True (đúng), chương trình sẽ thực hiện các công việc. Quay lại bước 1.

+ Nếu BTĐK là False (sai), chương trình sẽ thoát khỏi vòng lặp while. Chuyển sang bước 3

Bước 3. Kết thúc.

*Ví dụ 4.3. In ra 1000 dấu: **

```
1  # khởi tạo biến _dem
2  _dem=0
3  while (_dem<=1000):
4      print("*") #in ra dấu *
5      _dem+=1    #tăng giá trị biến đếm lên 1: _dem= dem +1
6
```

Kết quả:

```
*
*
*
*
*
*
*
```

Vậy, giải quyết bài toán ở ví dụ 4.1 trở lên rất đơn giản như sau:

```
# khởi tạo biến _dem
_dem=1
while (_dem<=10000):
    print(_dem) #in ra giá trị biến đếm
    _dem+=1    #tăng giá trị biến đếm lên 1: _dem= dem +1

```

Kết quả:

```
9989
9990
9991
9992
9993
9994
9995
9996
9997
9998
9999
10000
```

Chú ý: Khi thiết lập biểu thức chúng ta tránh trường hợp để điều kiện luôn đúng dẫn đến chương bị lặp vô hạn

Ví dụ 4.4. Sẽ lặp vô hạn

```
# khởi tạo biến _dem
_dem=1
print("Bạn hãy nhập vào 1 số nguyên:")
_so = input()

while (_dem >=0):
    print(_dem) #in ra giá trị biến đếm
    _dem+=1    #tăng giá trị biến đếm lên 1: _dem= dem +1

```



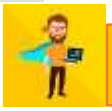
Bài thực hành:

+Yêu cầu người dùng nhập vào 1 số nguyên bất kỳ

+ Hiển thị các số từ 1 đến số vừa nhập




```
1   # khởi tạo biến _dem
2   _dem=1
3   print("Bạn hãy nhập vào 1 số nguyên:")
4   _so = input()
5
6   while (_dem<=int(_so)):
7       print(_dem) #in ra giá trị biến đếm
8       _dem+=1    #tăng giá trị biến đếm lên 1: _dem= dem +1
9
10
```



2. Giới thiệu lệnh break, continue

2.1. Lệnh Break

- **Tác dụng**: Lệnh break này có thể được sử dụng trong vòng lặp while và vòng lặp for (vòng lặp for các em sẽ tìm hiểu ở bài 5). Nếu bạn đang sử dụng trong vòng lặp, thì lệnh break kết thúc sự thực thi của vòng lặp bên trong và bắt đầu thực thi dòng code tiếp theo của khối**.**
- **Cú pháp:** lệnh break có cú pháp khá đơn giản break
- Ví dụ 4.5*:*

```
a = 10
while a > 0:
    print ('Gia tri bien hien tai la:', a)
    a = a -1
    if a == 4:
        break
print("Good bye!" )
```

Kết quả:

```
Gia tri bien hien tai la: 10
Gia tri bien hien tai la: 9
Gia tri bien hien tai la: 8
Gia tri bien hien tai la: 7
Gia tri bien hien tai la: 6
Gia tri bien hien tai la: 5
Good bye!
```

2.2. Lệnh Continue

- **Tác dụng**: Lệnh này bỏ qua lần lặp hiện tại và bắt buộc lần lặp tiếp theo của vòng lặp diễn ra. Lệnh continue có thể được sử dụng trong vòng lặp while hoặc vòng lặp for.

While (BTĐK):
Lệnh 1
Lệnh 2
continue
Lệnh 3

Gặp lệnh continue, python sẽ quay lại xét biểu thức điều kiện và không thực hiện lệnh 3

Cú pháp: continue

```
a = 10
while a > 0:
    a = a -1
    if a == 4:
      continue
    print ('Gia tri bien hien tai la:', a)
print("Good bye!" )
```

Kết quả

```
Gia tri bien hien tai la: 9
Gia tri bien hien tai la: 8
Gia tri bien hien tai la: 7
Gia tri bien hien tai la: 6
Gia tri bien hien tai la: 5
Gia tri bien hien tai la: 3
Gia tri bien hien tai la: 2
Gia tri bien hien tai la: 1
Gia tri bien hien tai la: 0
Good bye!
```



Xin chúc mừng các em đã hoàn thành xong bài số 4, trong này chúng ta cần nhớ một số nội dung quan trọng sau:

1. Câu lệnh while cho phép thực thi các phần mã được lặp lại nhiều lần.
2. Cú pháp lệnh while:

while (btđk):

Lệnh 1

Lệnh 2 ..

Lệnh n

BTĐK = true thì các lệnh 1, .., n được thực hiện; khi BTĐK = false vòng lặp while sẽ kết thúc.

3. Khi sử dụng while cần chú ý btđk tránh trường hợp lặp vô hạn
4. Lệnh break: sẽ kết thúc vòng lặp
5. Lệnh continue: khi gặp lệnh này, chương trình sẽ quay lại xét btđk của vòng lặp while

BÀI TẬP

Bài 1. Có bao nhiêu dấu hoa thị được in ra trong đoạn mã sau đây?

```
a = 0
while a < 100:
    print('*', end="")
    a += 1
```

Bài 2. Có bao nhiêu dấu hoa thị được in ra trong đoạn mã sau đây?

```
a = 0
while a < 100:
    if a % 5 == 0:
        print(a, "Chia hết cho 5")
    a += 1
```

Bài 3: Sử dụng vòng lặp while để hiển thị các hình sau:



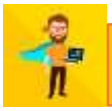
Bài 4. Nhập vào 1 số nguyên n bất kỳ. Tính tổng S, biết S = 1+ … + n

Bài 5. Bạn hãy nhập vào số nguyên n bất kỳ. Tính tổng các số lẻ từ 1 đến n. Chú ý: a là số lẻ nếu a % 2 = 1

BÀI SỐ 5. VÒNG LẶP XÁC ĐỊNH FOR

Nội dung bài học

- **Sử dụng vòng lặp xác định for trong Python**
- **Sử dụng vòng lặp for lồng nhau**
- **Ứng dụng vòng lặp for trong các ví dụ thực tiễn.**



1. Giới thiệu vòng lặp for

Chúng ta hãy xét ví dụ sau:

Ví dụ 5.1.

```
n = 1
while n <= 10:
    print(n)
    n += 1
```

Đoạn code trên lệnh print (n) sẽ thực hiện 10 lần, với 3 phần quan quan trọng để quản lý vòng lặp trên:

- Khởi tạo: n = 1
- Kiểm tra: n <= 10
- Cập nhật: n +=1

Với việc sử dụng vòng lặp While ở trên nếu các em không xử lý tốt có thể dẫn đến lặp vô hạn (như đã trình bày ở bài 4). Trong Python cung cấp thêm 1 vòng lặp xác định for giúp chúng ta xác định rõ vòng lặp sẽ thực hiện bao nhiêu lần.

Cú pháp vòng lặp for:

for <*biến*> **in range**(*giá trị bắt đầu, giá trị kết thúc, step*):

    công việc thực hiện

Giải thích: range (giá trị bắt đầu, giá trị kết thúc, step)

- Giá trị bắt đầu: giá trị đầu tiên gán cho <*biến*>
- Giá trị kết thúc: giá trị cuối cùng gán cho <*biến*>
- Step: thiết lập giá trị biến tăng lên hoặc giảm đi bao nhiêu, nếu step không ghi thì mặc định mỗi lần lặp sẽ tăng biến lên 1 đơn vị.

Ví dụ 5.1 ở trên được viết lại như sau: **for** *n* **in range** (*1,10*):

print(n)

ở đoạn mã này: n sẽ được gán lần lượt giá trị là từ 1 đến 10, tham số step không được ghi cho nghĩa là mỗi lần lặp n sẽ tăng lên 1.

Ví dụ 5.2

```
for n in range (21, 0, -2):
    print (n, end=" ")
```

Kết quả:

```
21 19 17 15 13 11 9 7 5 3 1
```

Các em chú ý: vì step = -2 **nên giá trị của n-3 sau mỗi lần lặp**

Ví dụ: Em hãy cho biết kết quả của các vòng lặp sau:

range(10) → thì sẽ in ra: 0,1,2,3,4,5,6,7,8,9

range(1, 10) → 1,2,3,4,5,6,7,8,9

range(1, 10, 2) → 1,3,5,7,9

range(10, 0, -1) → 10,9,8,7,6,5,4,3,2,1

range(10, 0, -2) → 10,8,6,4,2

range(2, 11, 2) → 2,4,6,8,10

range(-5, 5) → −5,−4,−3,−2,−1,0,1,2,3,4

range(1, 2) → 1

range(1, 1) → (empty)

range(1, -1) → (empty)

range(1, -1, -1) → 1,0

range(0) → (empty)



Xin chúc mừng các em đã hoàn thành xong bài số 5, trong này chúng ta cần nhớ một số nội dung quan trọng sau:

Cú pháp vòng lặp for:

for <*biến*> **in range**(*giá trị bắt đầu, giá trị kết thúc, step*):

công việc thực hiện

Giải thích: range (giá trị bắt đầu, giá trị kết thúc, step)

- Giá trị bắt đầu: giá trị đầu tiên gán cho <*biến*>

- Giá trị kết thúc: giá trị cuối cùng gán cho <*biến*>
- Step: thiết lập giá trị biến tăng lên hoặc giảm đi bao nhiêu, nếu step không ghi thì mặc định mỗi lần lặp sẽ tăng biến lên 1 đơn vị.

BÀI TẬP THỰC HÀNH

Bài 1. Cho biết kết quả của đoạn code sau:

```
for a in range(100):
    print('*', end='')
print()
```

Bài 2. Cho biết kết quả của đoạn code sau:

```
for a in range(20, 100, 5):
    print('*', end='')
print()
```

Bài 3. Cho biết kết quả của đoạn code sau:

```
for a in range(100, 0, -2):
    print('*', end='')
print()
```

Bài 4. Cho biết kết quả của đoạn code sau:

```
for a in range(1, 1):
    print('*', end='')
print()
```

Bài 5. Cho biết kết quả của đoạn code sau:

```
for a in range(-100, 100):
    print('*', end='')
print()
```

Bài 6: Sử dụng ngôn ngữ Python để viết code theo yêu cầu sau:

1. In ra các số chia hết cho 5 và nhỏ hơn 1000
2. In ra các số chia hết cho 5 và chia hết cho 7 mà lớn hơn 1000 và nhỏ hơn 10.000
3. In ra các số chẵn chia cho 7 dư 6 mà có 3 chữ số

Bài 7: Sử dụng ngôn ngữ Python để viết code theo yêu cầu sau:

1. Nhập vào 1 số nguyên bất kỳ
2. Tính tổng các số nhỏ hơn số đã nhập
3. Tìm các số lẻ nhỏ hơn số đã nhập
4. Tính tổng các số lẻ nhỏ hơn số đã nhập

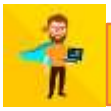
Bài 8: Sử dụng ngôn ngữ Python để viết code theo yêu cầu sau:

1. Nhập vào số tiền bạn muốn gửi
2. Nhập vào số năm bạn gửi
3. Tính số tiền bạn sẽ có sau khi gửi (biết số tiền năm sau = tiền năm trước + 12% * tiền năm trước)
4. Nếu mỗi năm rút ra 1/10 số tiền thì đến lúc hết hạn bạn sẽ có bao nhiêu tiền (biết số tiền năm sau = tiền năm trước + 12% * tiền năm trước)

BÀI SỐ 6. ÔN TẬP NỘI DUNG TỪ BÀI 1 ĐẾN BÀI 5 (2 tiết)

Nội dung bài học:

- **Ôn tập nội dung kiến thức từ bài 1 đến bài 5**
- **Ứng dụng vào 1 số bài tập thực hiễn**



1. Biến, kiểu dữ liệu, toán tử

1.1. Biến

Khái niện biến: *là các đối tượng sử dụng để chứa giá trị nào đó và, giá trị đó có thể thay đổi trong suốt quá trình thực hiện chương trình* Khai báo biến: TenBien= GiatriBien

Để xóa biến dùng hàm: del <biến>

1.2. Kiểu dữ liệu

Các loại dữ liệu mà biến có thể chứa trong Python: số nguyên (int), số thực(float), chuỗi(str), số phức (complex), kiểu luận lý (bool).

Dùng hàm type() để kiểm tra kiểu dữ liệu của biến, ví dụ: type(*tên_biến*).

Có thể ép kiểu dữ liệu: float(), int(), str(data),… ví dụ: int(*ten_bien*).

1.3. Toán tử

Toán tử toán học:

Toán tử toán học	Mô tả	Ví dụ
+	Cộng	12 + 4.9 => kết quả 16.9
–	Trừ	3.98 – 4 => kết quả -0.02
*	Nhân	2 * 3.4 => kết quả 6.8
/	Chia	9 / 2 => kết quả 4.5

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

//	Chia lấy phần nguyên	9 // 2 => kết quả 4
%	Chia lấy phần dư	9%2 =>kết quả 1
**	Lũy thừa	3**4=>kết quả 81

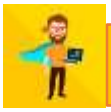
Toán tử quan hệ:

Toán tử quan hệ	Miêu tả
<	Nhỏ hơn.
>	Lớn hơn
<=	Nhỏ hơn hoặc bằng
>=	Lớn hơn hoặc bằng
==	Bằng
!=	Không bằng
<>	Không bằng (tương tự !=)

Toán tử gán:

Toán tử gán	Mô tả	Ví dụ	Tương đương với
=	Phép gán giá trị bên phải cho biến bên trái dấu bằng	x=5	
+=	Cộng và gán	x=2 x+=5 ==>x=7	x=x+5
-=	Trừ và gán	x=2 x-=5 ==>x=-3	x=x-5
*=	Nhân và gán	x=2 x*=5 ==>x=10	x=x*5

/=	Chia và gán	x=7 x/=5 ==>x=1.4	x=x/5
//=	Chia và gán (lấy nguyên)	x=7 x//=5 ==>x=1	x=x//5
%=	Chia lấy dư	x=7 x%=5 ==>x=2	x=x%5
**=	Lấy lũy thừa và gán	x=2x**=3 ==>x là 2 mũ 3 =8	x=x**3



2. Ghi chú lệnh, hàm print(), input()

2.1. Ghi chú lệnh

Ghi chú 1 dòng: Dùng từ khóa #

Ghi chú nhiều dòng: Dùng “”” nội dung cần ghi chú “”” (3 cặp nháy đôi) hoặc ”’ nội dung cần ghi chú ”' (3 cặp nháy đơn)

2.2. Hàm input()

Hàm input() thường là kiểu chuỗi, do đó ta cần chuyển kiểu nếu như muốn lưu trữ giá trị nhập vào không phải kiểu chuỗi.

2.3. Hàm print()

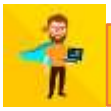
- Hàm print() với mục đích xuất dữ liệu ra màn hình trên các dòng khác nhau
- Khi chúng ta muốn các thông tin hiển thị trên cùng 1 dòng dữ liêu, bằng cách thêm đối số *end*= ‘ ‘ vào trong hàm print, vd: print(“*”, end=’ ‘)
- Ngoài ra Python còn hỗ trợ một số ký tự đặc biệt khi xuất dữ liệu ra màn hình, đó là:

\n: xuống dòng

\t: đẩy vào 1 tab

\’ hoặc \”: xuất trích dẫn

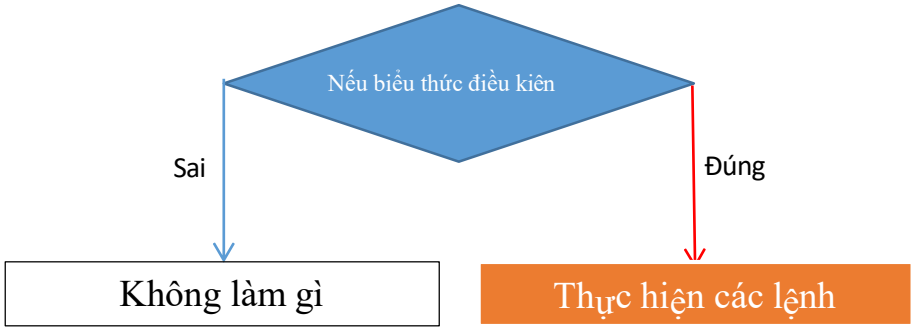
- Hàm print là cho phép *lặp một chuỗi nào đó liên tục* với số lần tùy chọn, bằng cách dùng phép nhân *, ví dụ: print(‘---’*30)



3. Cấu trúc điều kiện

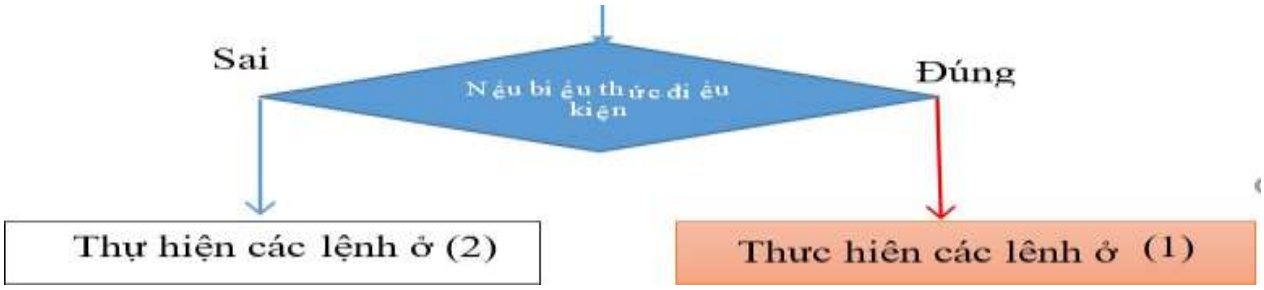
3.1. cấu trúc if

```
if bieu_thuc:
    cac_lenh
```



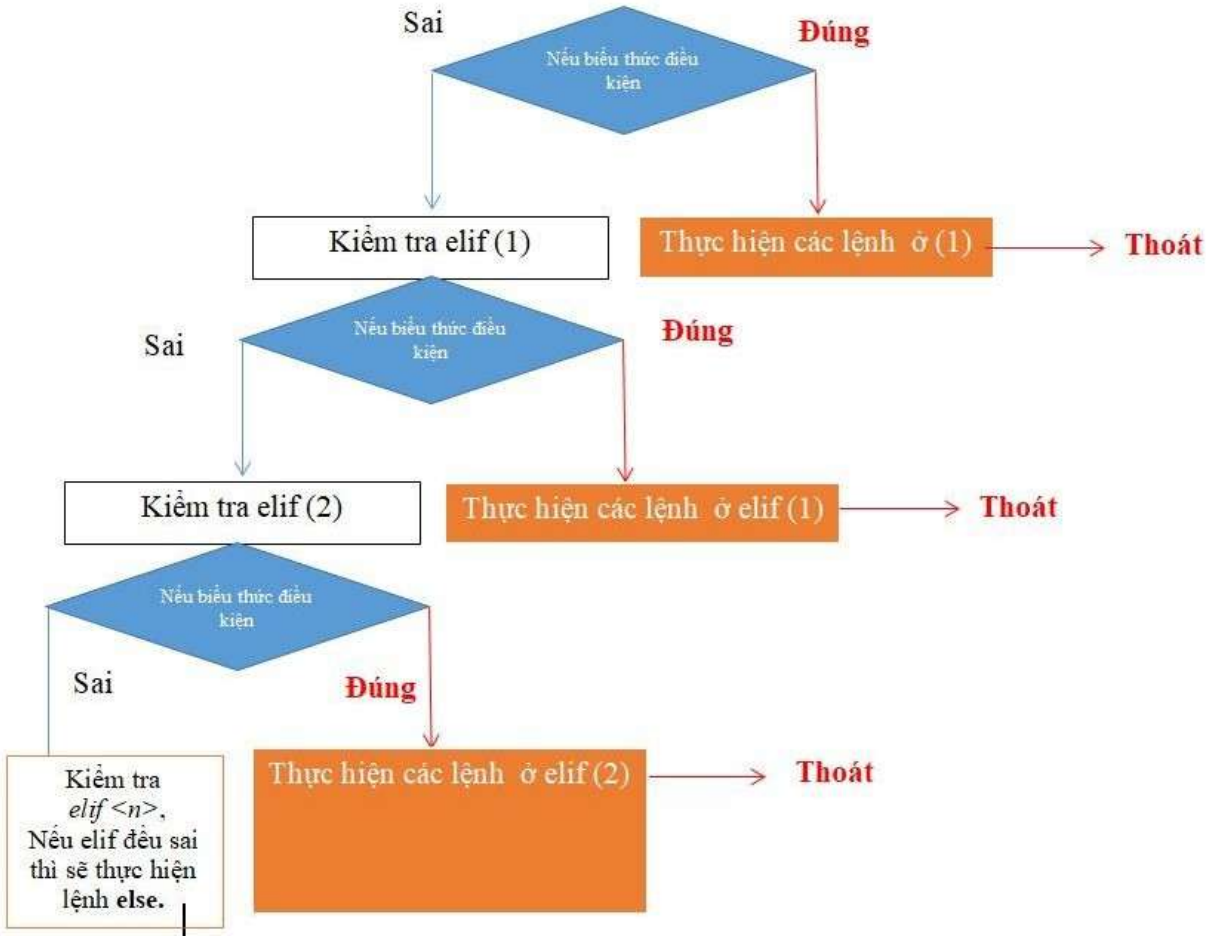
3.2. Cấutrúc if… else….

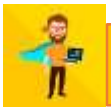
```
if bieu_thuc:
   cac_lenh      1
else:
   cac_lenh      2
```



3.3. Cấu trúc if… elif…..else….

```
if bieu_thuc1:
        cac_lenh (1)
elif bieu_thuc2:
        cac_lenh (2)
elif bieu_thuc3:
        cac_lenh (3)
else:
        cac_lenh (4)
```





4. Ghi chú lệnh, hàm print(), input()

Cú pháp vòng lặp while:

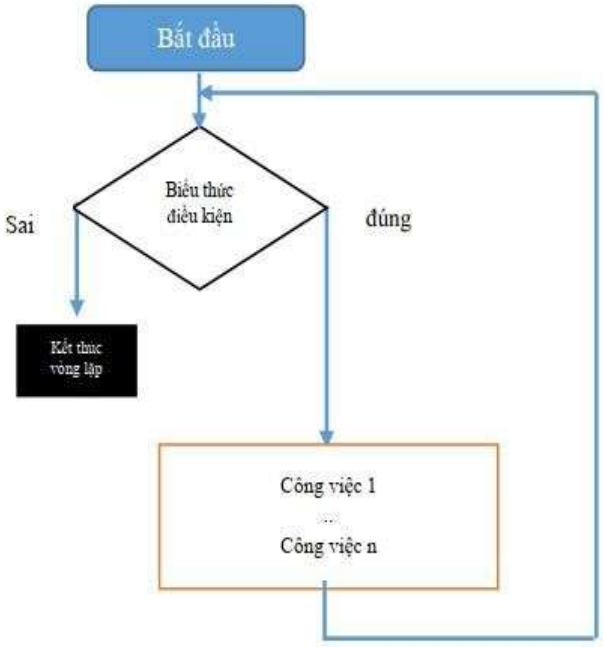
while (*biểu thức điều kiện*):

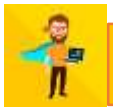
Công việc 1

Công việc 2

Công việc n

Sơ đồ thực hiện lệnh while như sau:





5. Ghi chú lệnh, hàm print(), input()

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Cú pháp vòng lặp FOR:

for <*biến*> **in range**(*giá trị bắt đầu, giá trị kết thúc, step*):

công việc thực hiện

Ví dụ: **for** *n* **in range** (*21, 0, -3*):

print (n, '', end='')

Với ví dụ trên, nếu trong range viết thành:

range(10) → thì sẽ in ra: 0,1,2,3,4,5,6,7,8,9
range(1, 10) → 1,2,3,4,5,6,7,8,9
range(1, 10, 2) → 1,3,5,7,9
range(10, 0, -1) → 10,9,8,7,6,5,4,3,2,1
range(10, 0, -2) → 10,8,6,4,2
range(2, 11, 2) → 2,4,6,8,10
range(-5, 5) → −5,−4,−3,−2,−1,0,1,2,3,4
range(1, 2) → 1
range(1, 1) → (empty)
range(1, -1) → (empty)
range(1, -1, -1) → 1,0
range(0) → (empty)



BÀI TẬP THỰC HÀNH

Bài 1. Phần mềm quản lý học lực

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

- Tạo giao diện phần mềm như sau:

```
* ********************************** *
* PHẦN MỀM QUẢN LÝ ĐIỂM VER 1.0      *
* Create by: Nguyễn văn A                    *
* ********************************** *
Nhấn phím: 1 để tiếp tục| nhấn phím: 0 để thoát
```

- Nhập điểm tổng kết của 3 môn: Toán, Văn, tiếng Anh
- Tính điểm tổng kết trung bình của 3 môn trên
- Đưa ra học lực:
  - Nếu điểm TBC>=9, in ra nhận xét: Bạn rất xuất sắc
  - Nếu điểm TBc>=8.0 và <9: in ra nhận xét: Đạt học sinh Giỏi
  - Nếu điểm TBc>=7 và <8 in ra nhận xét: Bạn học khá ▪ Nếu điểm TBc>=5 và <7 in ra nhận xét: Bạn học trung bình ▪ Nếu điểm TB<5 in ra nhận xét: Bạn học kém.

Bài 2. MINI GAME

Máy tính khởi tạo 1 số ngẫu nhiên bất kỳ từ 1 – 300. Người chơi sẽ phải đoán ra số đó; Người chơi chỉ được đoán 10 lần, trong mỗi lần người chơi đưa ra kết quả, thì máy tính sẽ kiểm tra xem kết quả và thông báo số đó lớn hơn hoặc nhỏ hơn hay bằng số mà máy tính đưa ra.

Nếu người chơi đoán đúng chương trình kết thúc; nếu sau 10 lần đoán mà chưa tìm được đáp án thì hiển thị thông báo người chơi thua cuộc.

Gợi ý làm bài: Để tạo 1 số ngẫu nhiên trong khoảng 1 tới 300 các em hàm: random.randint(1,300)

Bài 3. Viết chương trình sắp xếp 3 số nhập vào và sắp xếp theo thứ tự tăng dần và giảm dần.

Bài 4. Phần mềm quản lý bán hàng

Em hãy viết một chương trình quản lý bán hàng với python theo yêu cầu như sau:

1. Tạo menu chứa: Tên các sản phẩm, giá thành để người chơi lựa chọn
2. Người chơi lựa chọn các loại sản phẩm với số lượng cụ thể.
3. Hiển thị hóa đơn thành toán của khách hàng bao gồm:

- Tên các sản phẩm với số lượng và thành tiền từng loại sản phẩm - Tổng số tiền phải trả.

Yêu cầu: *Chương trình có giao diện đẹp, dễ sử dụng.*

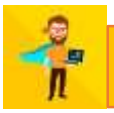
BÀI SỐ 8. TÌM HIỂU VỀ CHUỖI (STRING) TRONG PYTHON

Nội dung bài học

- Tìm hiểu về chuỗi trong Python

- Ứng dụng vào 1 số bài tập thực hiễn

- Các hàm trong List



1. Tìm hiểu về chuỗi

Trong thuật ngữ lập trình, chúng ta thường gọi văn bản là một *chuỗi* (string). Để đơn giản, các em cứ nghĩ *chuỗi* là bộ sưu tập các chữ cái. Ví dụ: tên, địa chỉ nhà của các em là một chuỗi. Trên thực tế, chương trình Python đầu tiên mà chúng ta tạo ra trong bài 1 đã sử dụng một chuỗi: "*Hello World*"

Trong Python, String được lưu giữ dưới dạng các ký tự đơn trong vị trí ô nhớ liên tiếp nhau.

H	E	L	L	0

Lợi thế của sử dụng chuỗi là:
+ Có thể được truy cập từ cả hai hướng (tiến về trước hoặc ngược về sau). Để truy cập các giá trị trong String, bạn sử dụng **dấu ngoặc vuông [ ]** có chỉ mục ở bên trong. *Ví dụ*:

```
var1 = "Hello World!"
print ("var1[0]: ", var1[0])
| | Kết quả: H
```

+ Giúp cho người lập trình xử lý ký tự một cách dễ dàng hơn

Cách sử dụng chuỗi như thế nào:

- **Khai báo: ten_bien = "*chuỗi ký tự*" hoặc ten_bien= '*chuỗi ký tự* '**

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Ví dụ:

FPT = ”Chao cam em”

FPT2 = ‘Chao cac em FPTER’

- **Truy cập các phần tử trong chuỗi:**

- Truy cập 1 phần tử: **Tên_chuỗi**[chỉ mục]

FPT2 = ‘Chao cac em FPTER’

FPT2[0] => kết quả: C

FPT2[7] = >kết quả: a

- Truy cập nhiều phần tử:

+ ten_chuoi[start : end:step]

+ ten_chuoi[:end:step]

+ ten_chuoi[start::step]

Lưu ý:

Nếu step > 0: lấy đến giá trị **end -1**

Nếu step < 0: Lấy đến giá trị **end + 1**

Ví dụ: FPT2 = ‘Chao cac em FPTER’

FPT2[0:5] => kết quả: *Chao*

FPT2[:5:2] => kết quả: Ca

FPT2[:8:-1] => kết quả: RETPF em

Nối chuỗi

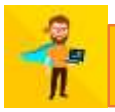
Có hai loại toán tử cơ bản có thể được sử dụng với String, đó là toán tử nối chuỗi + và toán tử lặp chuỗi *.

Toán tử nối chuỗi + được sử dụng để nối hai chuỗi với nhau và tạo nên một chuỗi mới.

Ví dụ: >>> “FPT ” + “School” ➔ Kết quả: FPT School

Toán tử lặp chuỗi* sử dụng hai thamsố. Một tham số là giá trị nguyên và thamsốkháclàchuỗi. Toántửlặpchuỗinàyđượcsửdụngđểlặpđilặplạimột chuỗi một số lần nào đó.

Ví dụ: >>> 5*”FPT” ➔ Kết quả: FPTFPTFPTFPTFPT



2. Các hàm thông dụng trong String (Quan

a. **capitalize():** Hàm này có tác dụng in hoa chữ cái đầu tiên của chuỗi.

```
Name = “fpt school”
print(Name.capitalize())
# kết quả: Fpt School
```

b. **center**

Cú pháp: string.center(len,char)

Trong đó:

string là chuỗi các bạn cần chuyển đổi.

len là số lượng ký tự của chuỗi mới.

char là ký tự sẽ hiển thị ở 2 bên chuỗi cũ. Mặc định nó sẽ là khoảng trắng.

Ví dụ:

```
1   name = "FPT School"
2   print(name.center(20))
3   # Kết quả:  FPT School
4   name = "FPT School"
5   print(name.center(20,"*"))
```

```
     FPT School
*****FPT School*****
```

c. **count() :** Hàm có tác dụng đếm xem trong chuỗi có bao nhiêu ký tự cần tìm
Cú pháp: string.count(sub)
Trong đó:
sub là chuỗi các bạn cần tìm kiếm và đếm.

Ví dụ:

```
name = "FPT School FPT"
print(name.count("FPT"))
```

```
2
```

d. **find():** Hàm này có tác dụng tìm kiếm một chuỗi trong một chuỗi hoặc khoảng chuỗi. Nó sẽ trả về là vị trí bắt đầu của chuỗi tìm được trong chuỗi nếu tìm thấy và nếu không tìm thấy nó sẽ trả về -1. Tương tự như hàm find(), hàm rfind() nhưng hàm này nó sẽ trả về index của chuỗi cuối cùng tìm được trong chuỗi. Cú pháp sử dụng tương tự hàm find().
Cú pháp: string.find(str)
Trong đó: str là chuỗi cần tìm
Ví dụ:

```
name = "FPT School FPT"
print(name.find("School"))
```

```
4
```

e. **index() :** Hàm này tương tự như hàm find() chỉ khác duy nhất là nếu như không tìm thấy thì hàm này sẽ gọi exception.

```
name = "FPT School FPT"
print("vi trị của SChool",name.index("School"))
print(name.index("AAA"))
```

```
vi trị của SChool 4
Traceback (most recent call last):
  File "main.py", line 3, in <module>
    print(name.index("AAA"))
ValueError: substring not found
```

f. **isalpha():** Hàm này có tác dụng kiểm tra xem một chuỗi có phải là chứa duy nhất các ký tự chữ hay không? Nó sẽ trả về True nếu chuỗi này chỉ chứa duy các ký tự chữ trong bảng chữ cái, và sẽ trả về False nếu nó chứa số hoặc ký tự đặc biệt khác.

```
name = "FPTSchool"
print(name.isalpha())
name = "FPTSchool2021"
print(name.isalpha())
```



g. **isdigit():** Hàm này có tác dụng kiểm tra xem một chuỗi có phải là chứa duy nhất các chữ số hay không? Nó sẽ trả về True nếu chuỗi này chỉ chứa duy các ký tự chữ và sẽ trả về False nếu nó chứa số hoặc ký tự đặc biệt khác.

```
name = "234567543"
print(name.isdigit())
name = "FPT 2021"
print(name.isdigit())
```



h. **lower():** Hàm này có tác dụng chuyển đổi chuỗi về dạng in thường.

i. **upper():** Hàm chuyển đổi chỗi về dạng in hoa

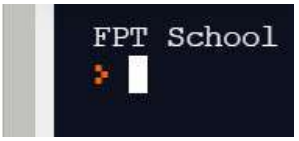
j. **capitalize():** Hàm chuyển đổi chuỗi về dạng in hoa những kỹ tự đầu tiên của mỗi từ

k. **strip(char):** Loại bỏ các ký tự char ở đầu 2 chuỗi, rstrip(char) loại bỏ các ký tự char ở phía bên phải chuỗi, lstrip() loại bỏ các ký tự char ở bên trái chuỗi, mặc định char là dấu cách

l. **replace():** Hàm này có tác dụng tìm kiếm và thay thế chuỗi tìm được bằng chuỗi mới.

Cú pháp: string.replace(old,new) với old là giá trị cũ, new là giá trị mới

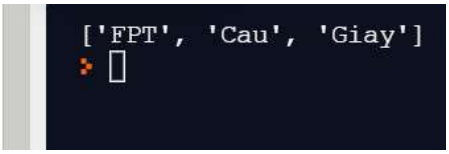
```
name = "FPT Cau Giay"
print(name.replace("Cau Giay", "School"))
```



m. **split() :** Hàm này có tác dụng tách chuỗi thành list bởi các char.

Cú pháp: string.split(char)

```
name = "FPT-Cau-Giay"
list1 = name.split("-")
print(list1)
```



BÀI TẬP THỰC HÀNH

Bài 1. Lập trình đọc vào 1 câu (ít hơn 30 ký tự) từ bàn phím rồi đếm xem trong câu có mấy từ (mỗi từ được cách nhau bởi dấu cách).

Bài 2. 1 chuỗi là 1 chuỗi dạng chuẩn, nếu đầu và cuối chuỗi không có dấu cách đồng thời trong chuỗi không có hai dấu cách kề nhau. Lập trình đọc vào 1 chuỗi và đưa ra ở dạng chuẩn. Đưa cả hai chuỗi ra màn hình.

Bài 3 Lập trình chương trình:

a. Đọc từ bàn phím một chuỗi
b. Đọc một kí tự từ bàn phím rồi đếm số lượng ký tự này trong chuỗi, thông báo kết quả ra mình hình.

Bài 4. Lập trình đọc vào một chuỗi. Không dùng chuỗi trung gian, hãy đảo các ký tự trong chuỗi theo trình tự ngược lại. Đưa ra chuỗi ban đầu và kết quả ra màn hình.

Ví dụ: Chuỗi ban đầu: 1234*

Kết quả: *4321

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

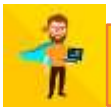
BÀI SỐ 9. LIST TRONG PYTHON

Nội dung bài học

- Tìm hiểu về List

- Vận dụng kiến thức về list và vòng lặp để lập trình ứng dụng

- Các hàm trong List



1. Tìm hiểu về List trong Python

List trong Python: Là một dạng dữ liệu cho phép lưu trữ nhiều kiểu dữ liệu khác nhau trong nó, và chúng ta có thể truy xuất đến các phần tử bên trong nó thông qua vị trí của phần tử đó trong list.

Ví dụ: ***List1*** **= [‘FPT’, 34, 8.09]**

Trong ví dụ trên: phần tử 1 là kiểu chuỗi, phần tử 2 là kiểu số nguyên, phần tử 3 là kiêu số thực.

Khởi tạo List

Một List gồm các yếu tố sau:

- Được giới hạn bởi cặp ngoặc **[ ]**, tất cả những gì nằm trong đó là những phần tử của List.
- Các phần tử của List được phân cách nhau ra bởi dấu phẩy (**,**).
- List có khả năng chứa mọi giá trị, đối tượng trong Python.

Cú pháp khái báo:

<ten_list> = [giatri1, giatri2, ..., giatriN]

Truy cập các giá trị trong List

Để truy cập các giá trị trong List, bạn sử dụng cú pháp sau:

```
<ten_list>[index]
```

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Index: Các phần tử trong một list được đánh dấu bắt đầu từ 0 theo chiều từ trái sang phải và từ -1 theo chiều từ phải qua trái

Ví dụ:



list1 = ['FPT', 'School', 2017, 2021]

list2 =[1,2,3,4,5,6,7]

list1[0]: 'FPT'

list2[2:5]: [3, 4, 5]

Lưu ý: Nếu step > 0: lấy đến giá trị **end -1**

Nếu step < 0: Lấy đến giá trị **end + 1**

Cập nhập giá trị của List

Chúng ta có thể cập nhật một hoặc nhiều phần tử của List bởi gán giá trị cho chỉ mục cụ thể đó. Cú pháp:

ten_list[index]=<giatri>

Ví dụ:
List1 = ['FPT', 'School', 2017, 2021]
List1[2] = 2020
⇨ List1 = ['FPT', 'School', 2020, 2021]

Các phép toán: + và * trong List

- Phép +: Ghép 2 list vào

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

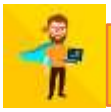
```
list1=[10,20]
list2=[30,40]
list3=list1+list2
print list3
```

Kết quả: [10, 20, 30, 40]

- Phép *: Nhân số phần tử lên

```
list1=[10,20]
print list1*2
```

Kết quả: [10, 20,10,20]



2. List lồng nhau

Do list có thể chứa nhiều kiểu dữ liệu khác nhau lên chúng ta hoàn toàn có thể khai báo một list chứa một hoặc nhiều list khác nhau.

Ví dụ: ListA = [[2, 3, 4, 5], [“a”, 45, 6], [“Aa”, 5, “bc”]]

print(ListA[1]) = [2, 3, 4, 5]

Chúng ta có thể lồng list khác vào trong list như ví dụ trên. Đối với list lồng nhau như này, chúng ta truy cập đến các phần tử như bình thường, theo cấp từ ngoài vào trong.

Ví dụ: Muốn in ra số 45 trong ListA

print(ListA[1][1] = 45

Nối 2 mảng: phép cộng

Bạn có thể sử dụng toán tử + để nối giá trị của 2 mảng và tạo ra một mảng lớn có số lượng phần tử là tổng số lượng phần tử của 2 mảng con.

Ví dụ:

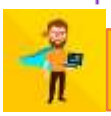
```
1  list1 = ["ab", 3, 5]
2  list2 = [4.5, -5, True]
3  list3 = list1 + list2
4  print(list1+list2)
```

```
['ab', 3, 5, 4.5, -5, True]
```

Phép nhân *: Nhân số phần tử lên

```
1  list1 = ["ab", 3, 5]
2  print(list1*3)
```

```
['ab', 3, 5, 'ab', 3, 5, 'ab', 3, 5]
```



3. Các hàm trong Python (Quan trọng)

3.1 Len(): Hàm này trả về số lượng phần tử có trong list.

Cú pháp: len(list)

Ví dụ: List1 = [“FPT”, “School”, 2021]

print(len(List1)) ➔ kết quả: 3

3.2 max(): Hàm này sẽ trả về phần tử có giá trị lớn nhất trong list. Nếu là chuỗi thì nó sẽ trả về phần tử có độ dài chuỗi dài nhất, nếu là số thì nó sẽ trả về phần tử có số lớn nhất.

Cú pháp: max(list)

Ví dụ:

```
list1 = ["ab", "cs", "s"]
print(max(list1))
list2 = [4, -7, 5.3, -2]
print(max(list2))
```

```
s
5.3
```

Nếu list có cả số lẫn ký tự thì không thể tìm được giá trị lớn nhất, chương trình báo lỗi

```
1  list3 = [4, -7, "b", "a"]
2  print(max(list3))
3
```

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(max(list3))
TypeError: '>' not supported between instances of 'str' and 'int'
```

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

3.3 min() : Hàm này sẽ trả về phần tử có giá trị nhỏ nhất trong list. Nếu là chuỗi thì nó sẽ trả về phần tử có độ dài chuỗi ngắn nhất, nếu là số thì nó sẽ trả về phần tử có số nhỏ nhất.

Cú pháp: min(list)

Ví dụ:

```
List1 = [5, -6, 8, -4]
print(min(List1))
List2 = ["Abc", "ds", "dfs"]
print(min(List2))
```

```
-6
Abc
```

Nếu list có cả số lẫn ký tự thì không thể tìm được giá trị lớn nhất, chương trình báo lỗi.

```
1  List3 = [5, -6,"Abc", "ds", "dfs"]
2  print(min(List3))
3
4
```

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(min(List3))
TypeError: '<' not supported between instances of 'str'
and 'int'
```

3.4 append(): Phương thức này có tác dụng thêm phần vào cuối của một list

Cú pháp: list.append(obj)

Trong đó: list là tên danh sách, obj là giá phần tử muốn thêm vào.

Ví dụ :

```
List3 = [5, -6,"ABC", "DE", "HF"]
print(List3)
List3.append("FPT")
print(List3)
```

```
[5, -6, 'ABC', 'DE', 'HF']
[5, -6, 'ABC', 'DE', 'HF', 'FPT']
```

3.3 extend() : Hàm này có tác dụng kế thừa lại các phần tử của list2 và thêm vào trong list1.

Trong đó:

list1 là list mà bạn muốn kế thừa từ một list khác (ở đây là list2).

list2 là list được sử dụng để cho list khác kết thừa (ở đây là list1).

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Ví dụ:

```
List3 = [5, -6,"ABC", "DE", "HF"]
print(List3)
List3.extend("FPT")
print(List3)
List3.extend(("School", 2021))
print(List3)
```

```
[5, -6, 'ABC', 'DE', 'HF']
[5, -6, 'ABC', 'DE', 'HF', 'F', 'P', 'T']
[5, -6, 'ABC', 'DE', 'HF', 'F', 'P', 'T', 'School', 2021]
```

Lưu ý: Về cơ bản thì extend() giống append() nhưng extend() có thể thêm nhiều hơn 2 phần tử cùng 1 lúc.

3.4 count() : Phương thức này có tác dụng đếm số lần xuất hiện của một thành phần trong list!

Cú pháp: list.count(item)

Ví dụ:

```
List3 = [5,"FPT", -6,"FPT"]
print(List3.count("FPT"))
```

```
2
```

3.5 index(): Phương thức này có tác dụng trả về index xuất hiện đầu tiên của phần tử mà bạn muốn tìm và nếu như không tìm thấy thì nó sẽ gọi exception.

Cú pháp: list.index(item)

Ví dụ:

```
List3 = [5,"FPT", -6,"FPT"]
print(List3.index("FPT"))
```

```
1
```

Trường hợp không có **item** sẽ báo lỗi **ValueError**

```
List3 = [5,"FPT", -6,"FPT"]
print(List3.index("School"))
```

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(List3.index("School"))
ValueError: 'School' is not in list
```

3.6 insert(): Phương thức có tác dụng thêm phần tử vào vị trí index của list, và các phần tử sau index đó sẽ được đẩy về phía sau.

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Cú pháp: list.insert(index, Item)

Trong đó:

List là list mà các bạn cần thêm.

Index là ví trí muốn chèn thêm item

Item là phần tử muốn thêm vào vị trí index

3.7 reverse(): Phương thức này có tác dụng đảo ngược vị trí của các phần tử trong list.

Ví dụ:

```
List1 = [5,"FPT", -6,"School"]
List1.reverse()
print(List1)
```

```
['School', -6, 'FPT', 5]
```

3.8 remove(): Phương thức này có tác dụng xóa phần tử khỏi list.

Cú pháp: list.remove(item)

Ví dụ:

```
List1 = [5,"FPT", -6,"School"]
print(List1)
List1.remove(-6)
print(List1)
```

```
[5, 'FPT', -6, 'School']
[5, 'FPT', 'School']
```

3.9 pop(): Phương thức này có tác dụng xóa bỏ phần tử trong list dựa trên index của nó. Nếu chỉ ghi pop() thì mặc định xóa phần tử cuối cùng.

Cú pháp: list.pop(index)

Ví dụ:

```
List1 = [5,"FPT", -6,"School", 2021]
print(List1)
List1.pop() #xóa phần tử cuối
print(List1)
List1.pop(1) # xóa phần tử 1
print(List1)
```

```
[5, 'FPT', -6, 'School', 2021]
[5, 'FPT', -6, 'School']
[5, -6, 'School']
```

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

3.10 sort(): Phương thức này có tác dụng sắp xếp lại các phần tử trong list theo một thứ tự xác định.

Cú pháp: list.sort(reverse)

Nếu reverse = True thì sắp xếp giảm dần, không ghi gì thì mặc định tăng dần

Ví dụ:

```
List1 = [5,-6, -4, 2021]
List1.sort()
print(List1)
```

```
[-6, -4, 5, 2021]
>
```

```
List1 = [5,-6, -4, 2021]
List1.sort(reverse = True)
print(List1)
```

```
[2021, 5, -4, -6]
>
```

3.10 clear(): Phương thức này có tác dụng xóa bỏ hết tất cả các phần tử trong list.

Cú pháp: list.clear()

Ví dụ:

```
List1 = [5,-6, -4, 2021]
print(List1)
List1.clear()
print(List1)
```

```
[5, -6, -4, 2021]
[]
>
```

Tuple

Tuple cũng là một cấu trúc mảng, tương tự như cấu trúc List. Một số điểm khác nhau cơ bản là khai báo Tuple sử dụng cặp dấu ngoặc (...) và một tuple đã được khai báo rồi thì không thay đổi được giá trị (immutable) và không hỗ trợ các phương thức như append() , pop() …Ví dụ:

```
1   tuple1 = (4, -7, "b", "a")
2   print(tuple1)
```

```
(4, -7, 'b', 'a')
>
```

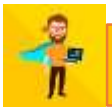
Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Vẫn hỗ trợ các cách để truy xuất phần tử giống List như là truy xuất theo index, range, tìm kiếm…

BÀI 10: TÌM HIỂU HÀM (FUNCTIONS) TRONG PYTHON

Nội dung:

- **Khái niệm hàm trong Python**
- **Tác dụng sử dụng hàm**
- **Cú pháp khái báo và định nghĩa hàm**
- **Cú pháp gọi hàm**
- **Tìm hiểu tham số, phạm vi biến trong hàm và ứng dụng**
- **Tìm hiểu từ khóa return và tác dụng**



1. Sử dụng hàm

Để khai báo một hàm trong Python thì chúng ta sử dụng từ khóa: **def** với cú pháp như sau:

```
def ten_ham ():
    #code ở đây

def ten_ham (tham số 1, tham số 2,…, tham số n):
        #code ở đây
```

Trong đó:

ten_ham: là tên của hàm mà các em muốn đặt. Lưu ý: Tên hàm không được bắt đầu bằng số và không được chứa các ký tự đặc biệt trừ ký tự _

tham số: là các giá trị mà các em muốn truyền vào hàm, nếu không có tham số thì để trống.

Ví dụ:

```
def SayFPT():
    print("Welcome to FPT")
def Pro(a, b):
    print("sum = " + str(a * b))
```

Cách gọi hàm

Để gọi một hàm đã được khai báo rồi, thì chúng ta sử dụng cú pháp sau:

Ten_ham() hoặc ten_ham(tham số 1, tham số 2, …, tham số n)

Trong đó:

+ *ten_ham* là tên của hàm là chúng ta muốn gọi.

+ *Tham số...* là các giá trị chúng ta muốn truyền vào trong hàm.

Ta gọi hàm **SayFPT()** đã được khai báo ở trong ví dụ trên

```
def SayFPT():
  print("Welcome to FPT")
SayFPT()
```

```
Welcome to FPT
>
```

Gọi hàm Pro(a,b)

```
def Pro(a, b):
  print("Pro = " + str(a * b))
Pro(5,6)
```

```
Pro = 30
>
```

Retun trong hàm: Cho phép lấy giá trị của 1 hàm trả về nhưng không hiển thị lên màn hình. Muốn hiển thị ta cần dùng print()

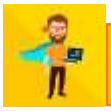
# khai báo hàm **def** tong(a, b): **return** a+ b	*giá trị trả về của hàm là*: 20

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

#---------- # *Gọi hàm.*` print("*giá trị trả về của hàm là*:", **tong(6,14)**)	

Khai báo hàm có giá trị mặc định
Khai báo hàm

Hàm không có tham số	**Hàm chứa tham số mặc định**
def sum(a,b): print(a+b) Gọi hàm: sum(6,7) → kết quả: 13 **Lưu ý** Nếu gọi**:** sum() → chương trình báo lỗi thiếu 2 biến	**def** sum(a= 10, b = 30): print(a+b) Gọi hàm: sum() ➔ kết quả : 30 sum(5,6) ➔ kết quả: 11
Cách dùng hàm có tham số mặc định sẽ tối ưu hơn	



2. Phạm vi biến của hàm

Khi một biến được khai báo ở trong hàm thì nó chỉ có thể được sử dụng ở trong hàm đó thôi.

```
def Hi_FPT():
    a = "Hello FPT"
    print(a)
print(a)
```

```
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    print(a)
NameError: name 'a' is not defined
```

Và chúng ta cũng không thể nào thay đổi giá trị của biến (biến bình thường) mà tác động ra ngoài hàm được.

Python định nghĩa:

+ Biến khai báo ở trong hàm gọi là biến **nội bộ (local variable)**

+ Biến khai báo ngoài hàm gọi là biến **toàn cầu (global variable)**

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Để khắc phục vấn đề trên, chúng ta sử dụng global variable - biến toàn cầu, khi một biến là global thì chúng ta có thể gọi và tác động đến nó từ bất kỳ đâu trong chương trình.

```
a = "FPT"
print(a)
def Hi_FPT():
  global a
  a = "Hello FPT"
  print(a)
Hi_FPT()
print(a) # giá trị a đã bị thay đổi
```

```
FPT
Hello FPT
Hello FPT
>
```

Truyền vô số tham số vào hàm

Trên thực tế, không phải lúc nào chúng ta cũng biết được chính xác số lượng biến truyền vào trong hàm. Chính vì thế trong Python có cũng cấp cho chúng ta khai báo một param đại diện cho các biến truyền vào hàm bằng cách thêm dấu * vào trước param đó.

```
def get_sum(*num):
    tmp = 0
    # duyet cac tham so
    for i in num:
        tmp += i
    return tmp
result = get_sum(1, 2, 3, 4, 5)
print(result)
```

```
15
>
```

Website tham khảo: https://toidicode.com/series/python-co-ban

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296

Giáo viên CNTT: Nguyễn Tài Sỹ
SĐT: 0967698296